

# Artificial Intelligence Computer Vision

Week 8, Term 2, 2026

Maryam Hashemi (m.Hashemi@unsw.edu.au)



**UNSW**  
SYDNEY

# Today's Roadmap

- **Introduction**
- Image Processing
  - Histogram Equalization
  - Noise Removal
  - Edge Detection
- Computer Vision
  - Convolutional Neural Networks
    - ✓ VGG16
    - ✓ AlexNet
- Cognitive Vision

# Why Is Vision Hard for Machines?



What is computer vision?



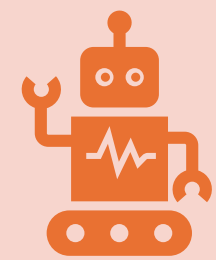
What is image processing?



What is the difference?



# From Pixels to Meaning



Computer vision is the field of AI that enables **machines to interpret and understand visual data** (images, videos) to perform tasks like object detection, image classification, and scene analysis.



Image processing involves applying algorithms to **transform or manipulate images, focusing on enhancing or modifying the visual content** (e.g., filtering, noise reduction).



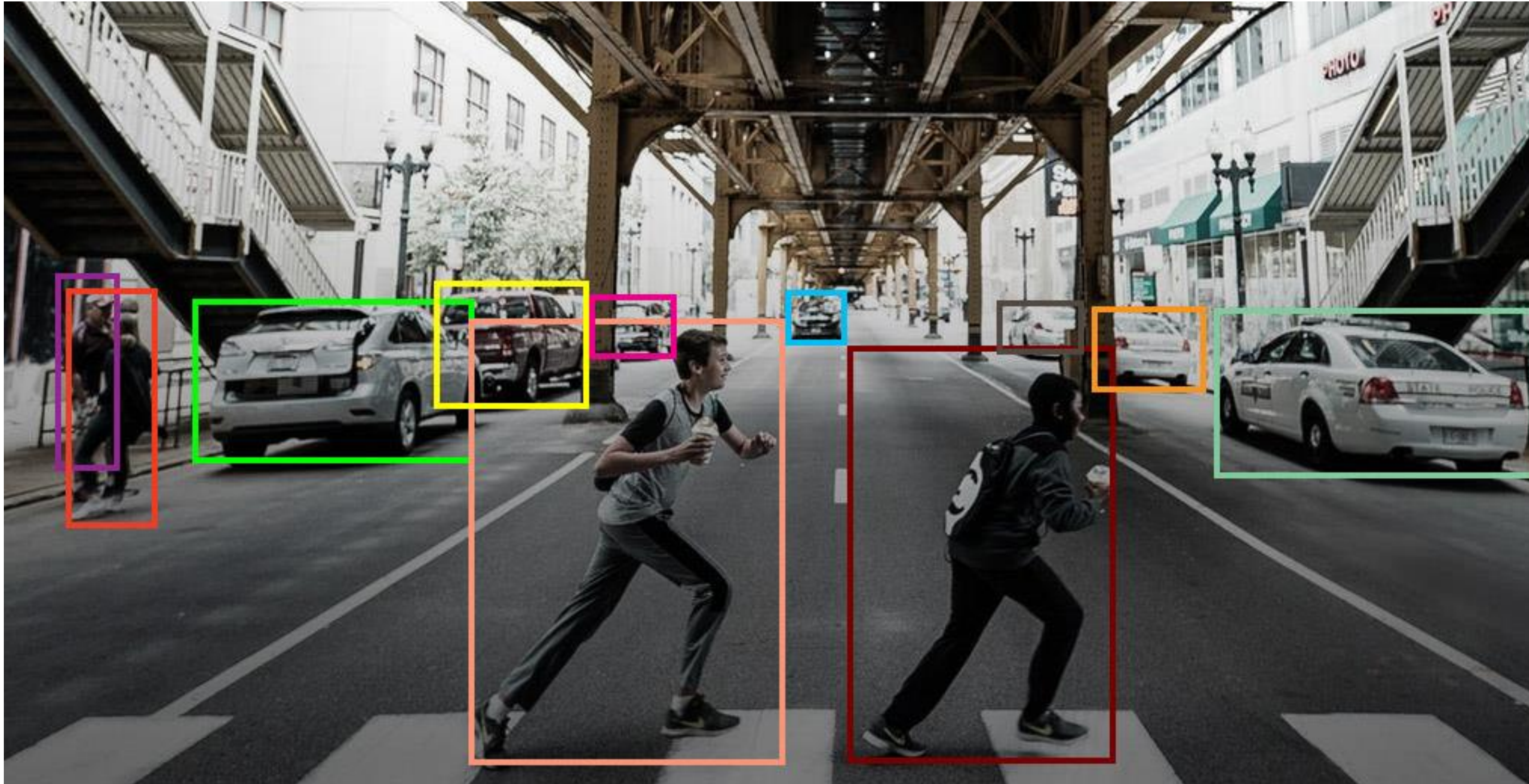
Computer vision aims to interpret and understand images, while image processing focuses on improving or transforming images.

CV is a subset of AI but image processing is a technique that might be used in AI approaches.



# Introduction

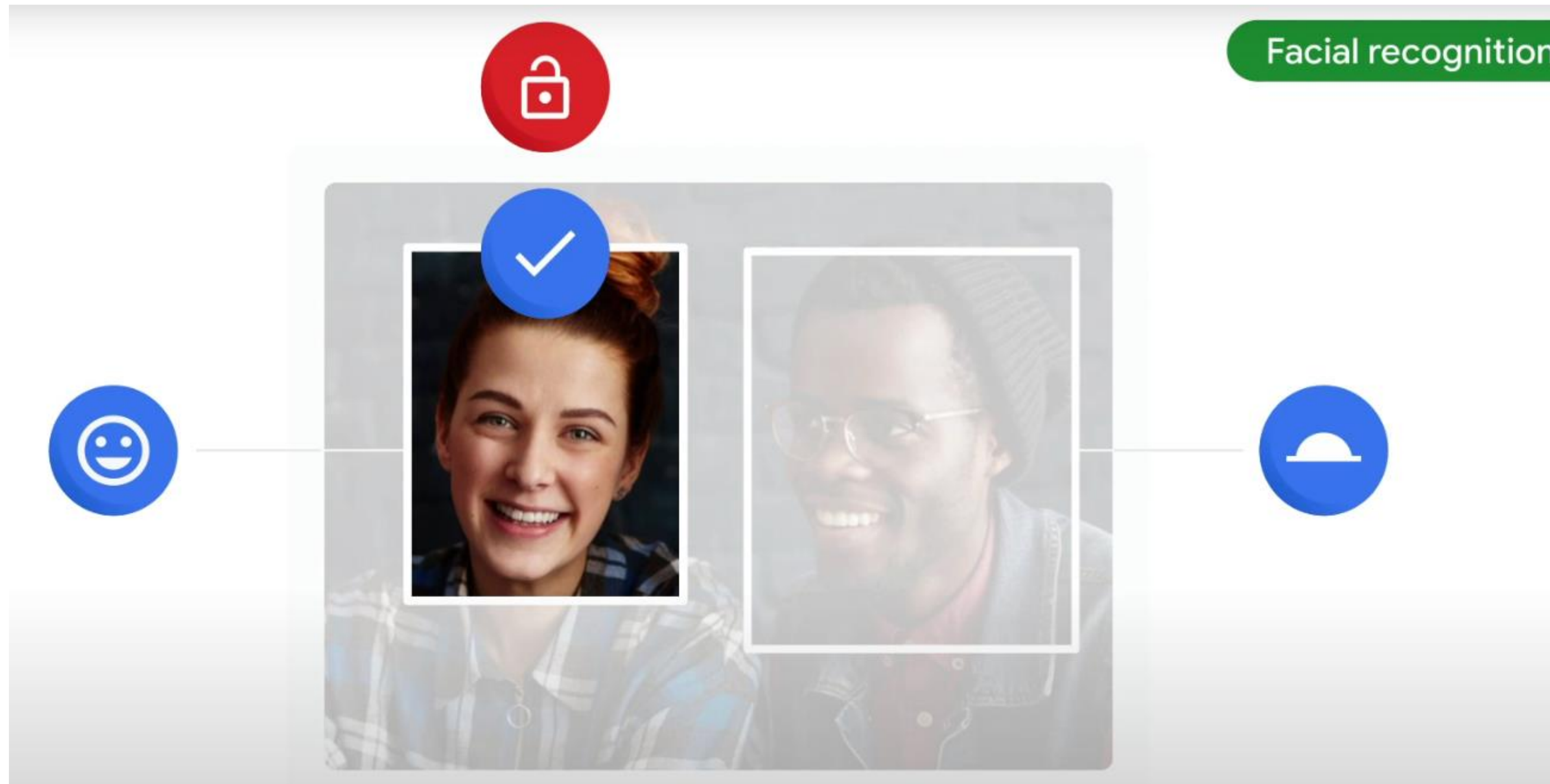
**Computer vision task:** Object detection and localization





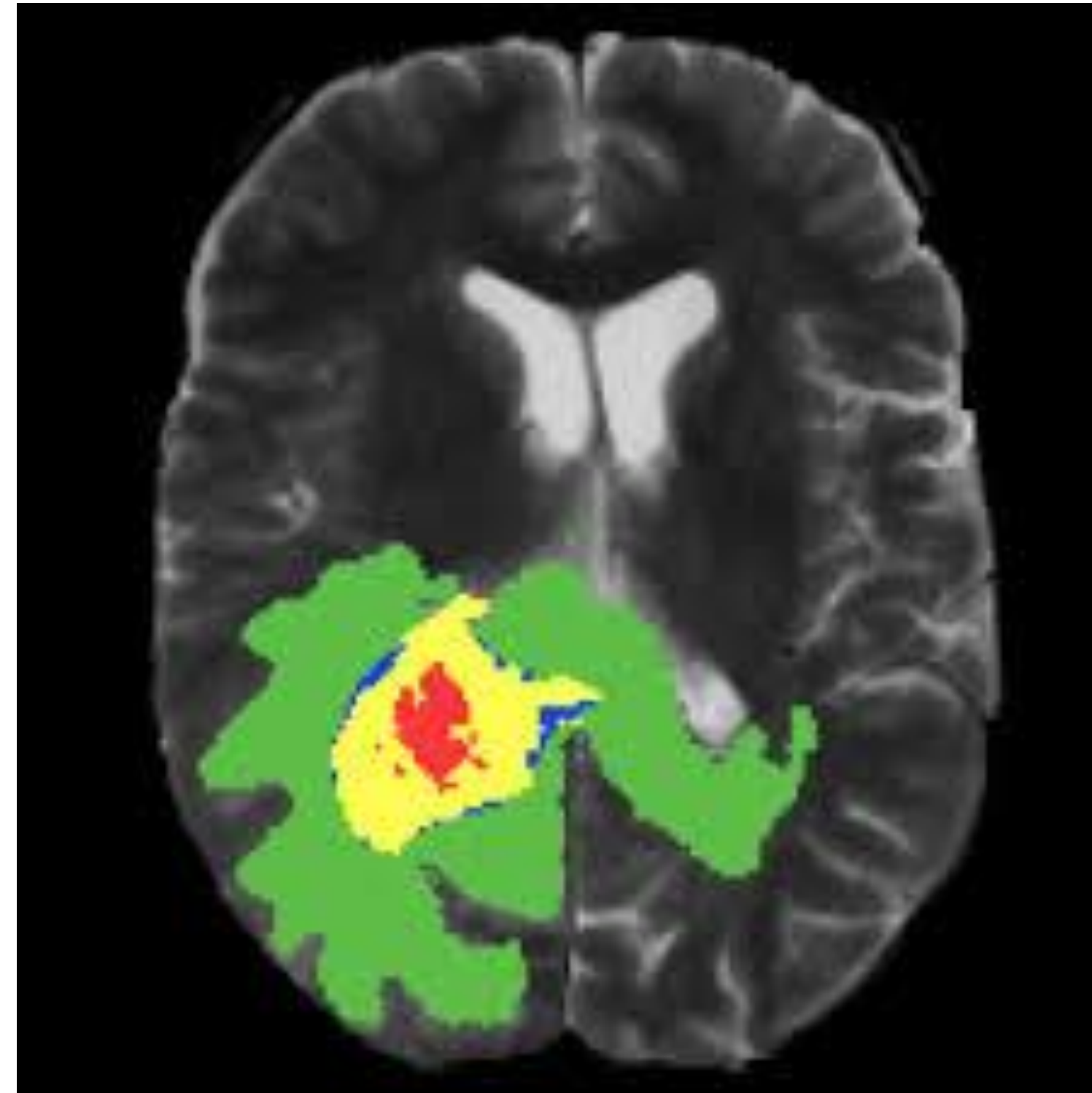
# Introduction

## Computer vision task: Facial Recognition



# Introduction

## Computer vision task: Segmentation





# Introduction

## Computer vision task: Classification



**Classification**

Cat



**Classification, Localization**

Cat



**Object Detection**

Cat, Dog



# Introduction

## Image processing task: Image Enhancement





# Introduction

Image processing task: Image Compression

**Original**



**5.7 MB**

**Compressed**



**470 KB**

# Why Human Vision Is Hard to Replicate?

Vision feels effortless to us, but it is difficult to reproduce computationally:

- Variable and uncontrolled illumination
- Shadows
- Complex and hard-to-describe objects
  - Objects from outdoor scenes
  - Non-rigid objects
  - Objects occluding other objects



# State of Computer Vision

- State of computer vision → The general computer vision problem remains unsolved
- Building a visual system as capable and general as human vision remains an open challenge after decades of research.
- Major progress has been made on specific computer vision problems:
  - e.g. face recognition used in digital cameras, surveillance, and security.
  - e.g. robotic pick-and-place.

# Computer Vision in Action

Computer vision in action



What we see

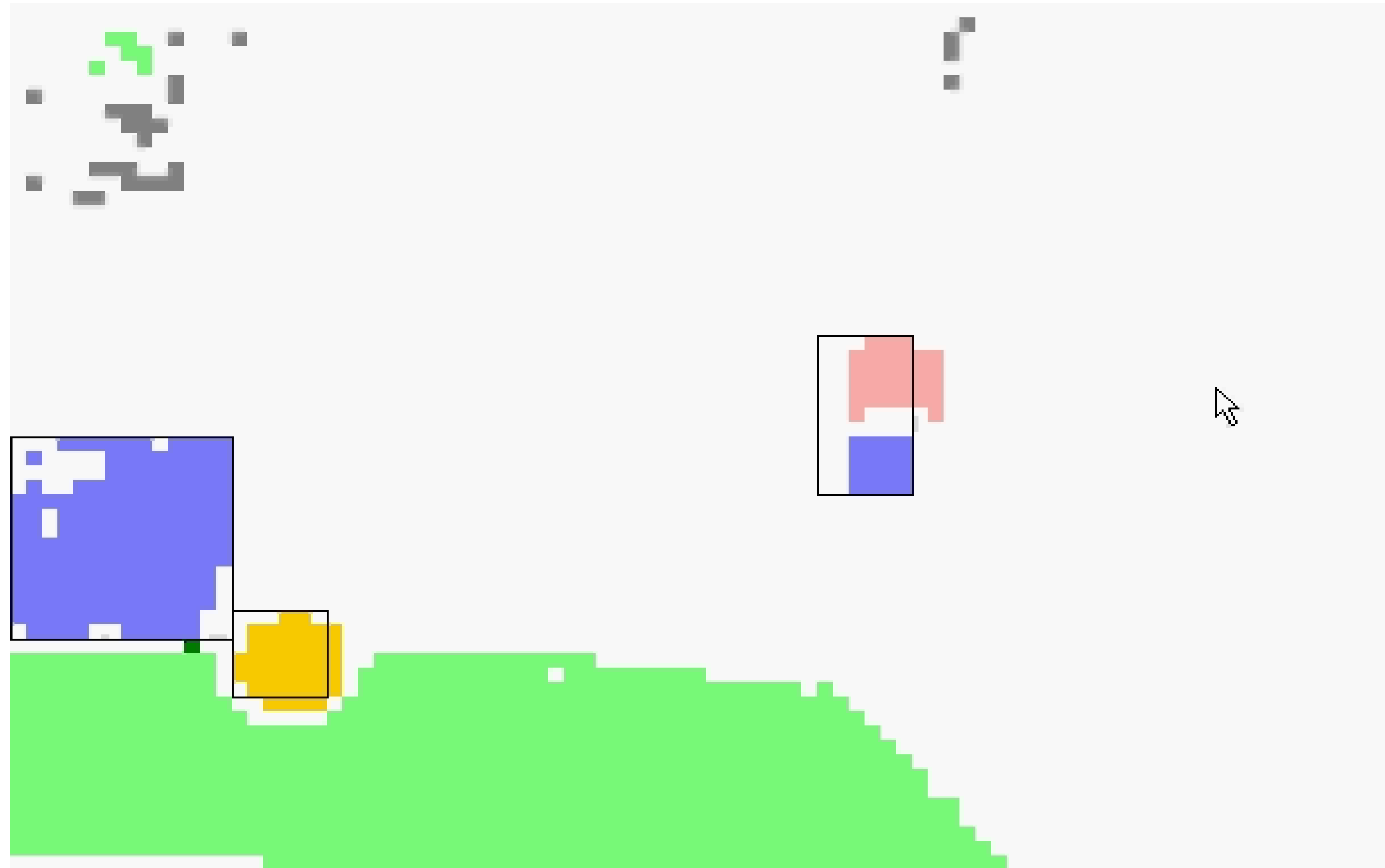


What we see from the robot's perspective



# What Does the Robot Actually See?

What the robot sees



# How Does Vision Support Action?

Computer vision in action



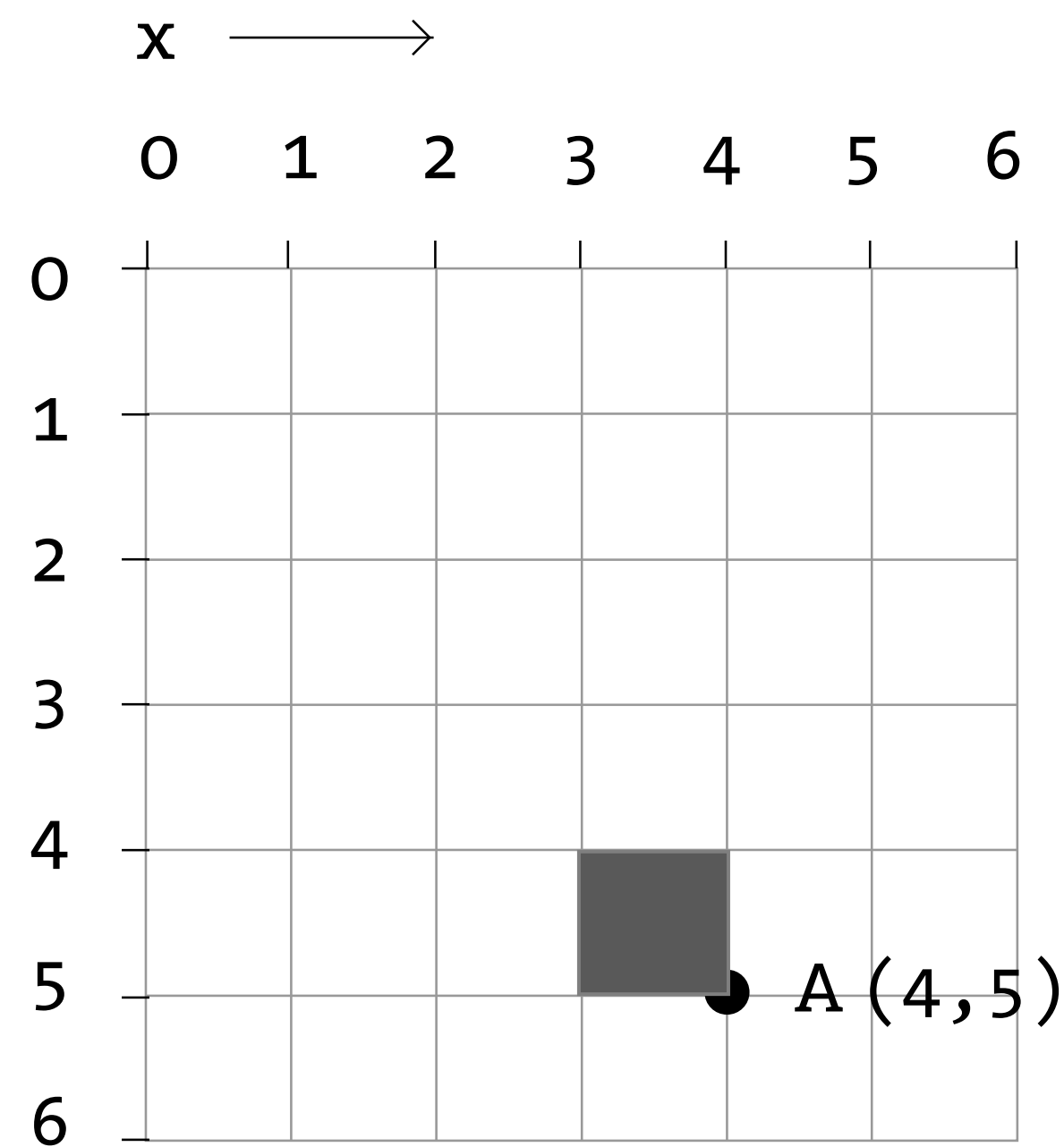


# Roadmap: Image Processing Foundations

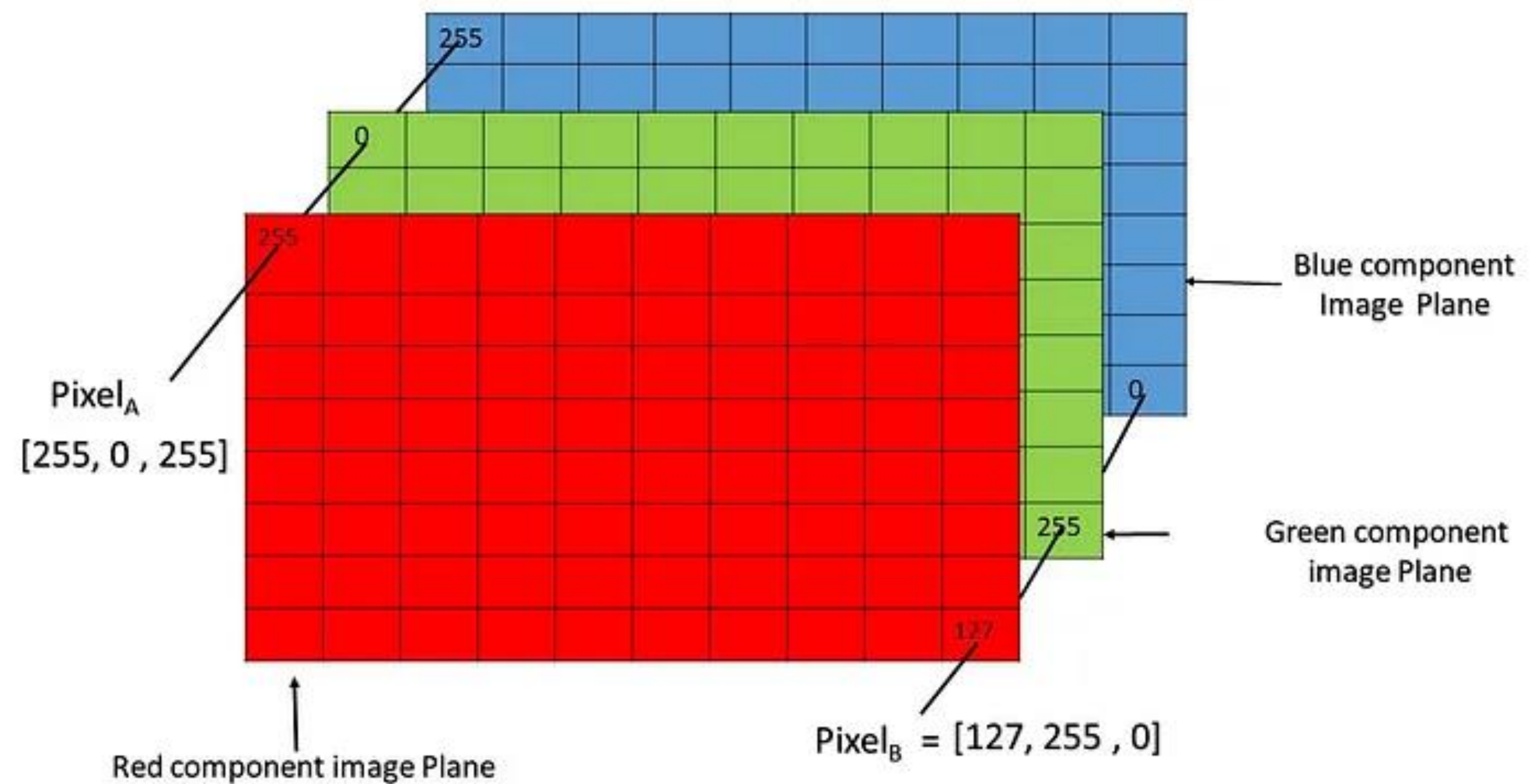
- Introduction
- **Image Processing**
  - Histogram Equalization
  - Noise Removal
  - Edge Detection
- Computer Vision
  - Convolutional Neural Networks
    - ✓ VGG16
    - ✓ AlexNet
- Cognitive Vision

# Images Are Arrays of Numbers

- A computer sees a digital image as a matrix in which each cell is a pixel.
- Each pixel stores a value that represents its intensity or color.



Example:  $A(4, 5) = 120$



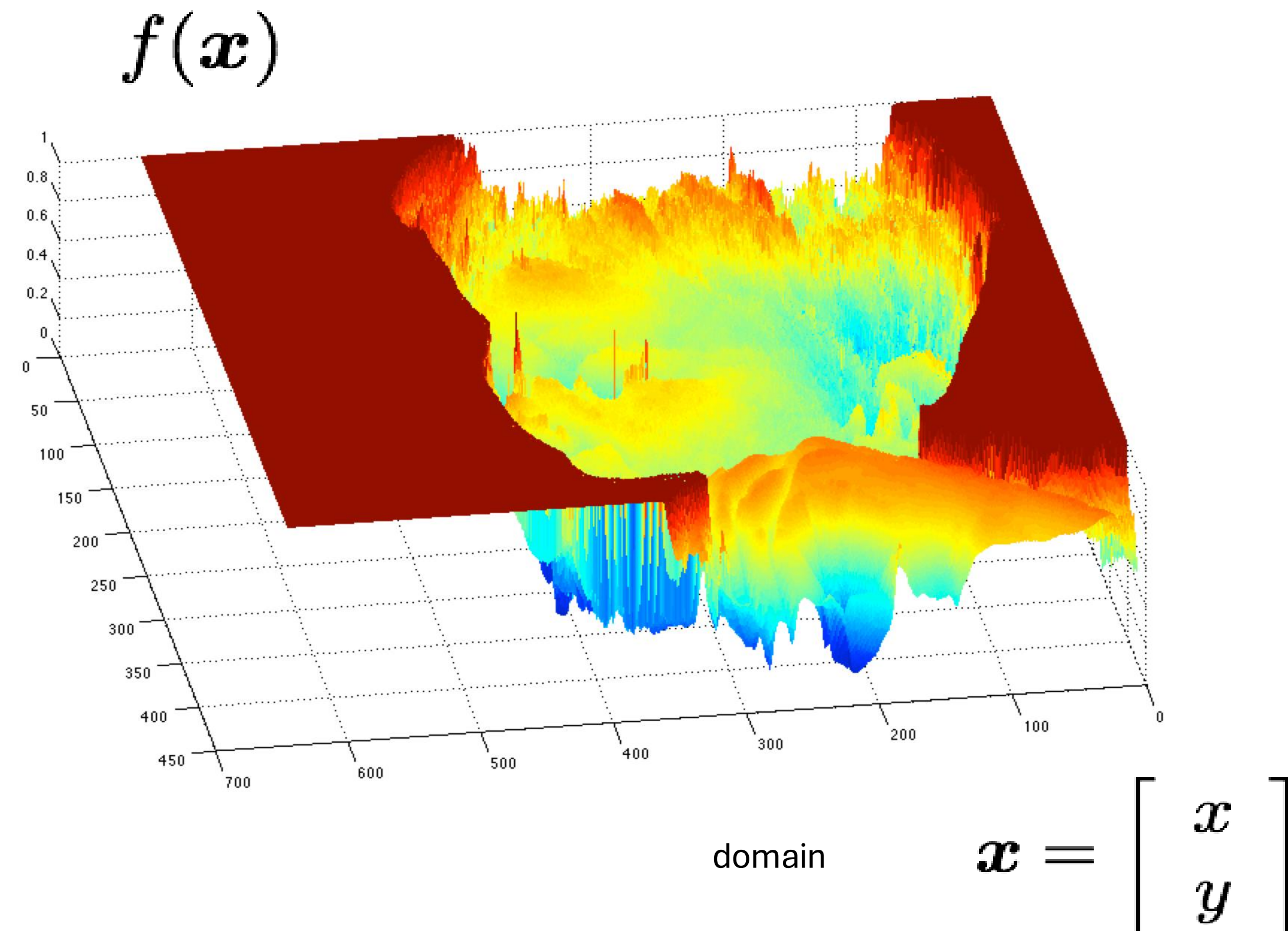


# Image Processing

- Image processing aims to enhance the information in an image that is most useful for later analysis.
- It is usually used to prepare images for further analysis and interpretation.
- An image can be represented as an  $n \times m$  array  $I(x,y) \rightarrow$  image intensity array.
- The cells are called pixels. Each number represents light intensity.
- We will discuss three important techniques: **histogram equalization**, noise removal, and edge detection.

# Image Processing: Histogram Equalization

- A grayscale image is a 2D function. In the example shown, we have 450 x 700 pixels, and each pixel can have a value between [0, 1] (or [0, 255] for an 8-bit representation).





# Image Processing: Histogram Equalization

- An image histogram is a plot of the gray-level frequencies (i.e., the number of pixels in the image that have that gray level).

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 2 | 0 |
| 1 | 0 | 7 | 7 | 7 | 0 |
| 0 | 7 | 0 | 0 | 7 | 0 |
| 1 | 0 | 0 | 7 | 2 | 0 |
| 0 | 0 | 7 | 1 | 0 | 1 |
| 1 | 0 | 7 | 7 | 7 | 0 |

Frequencies:

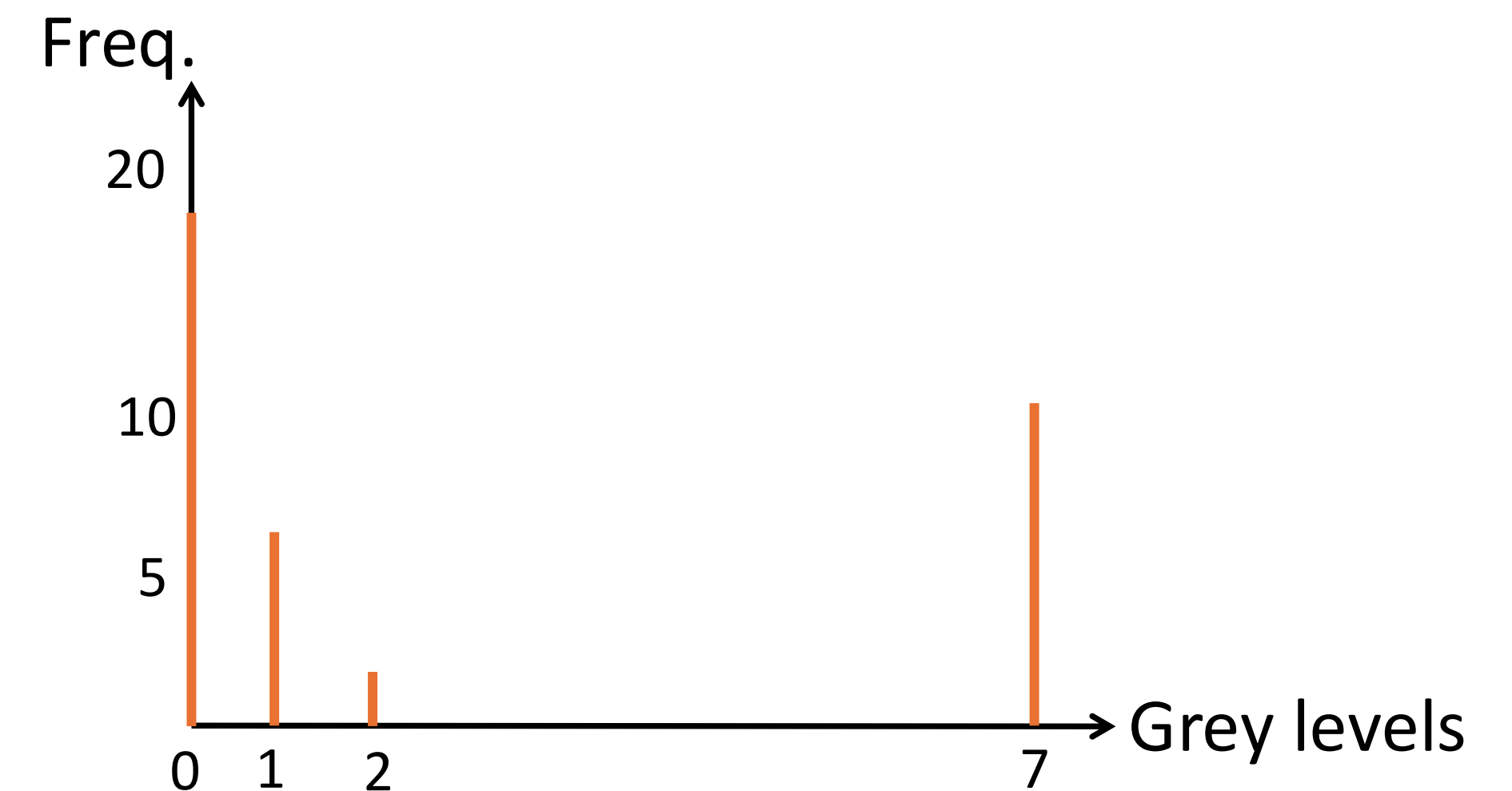
$$f(0) = 18$$

$$f(1) = 6$$

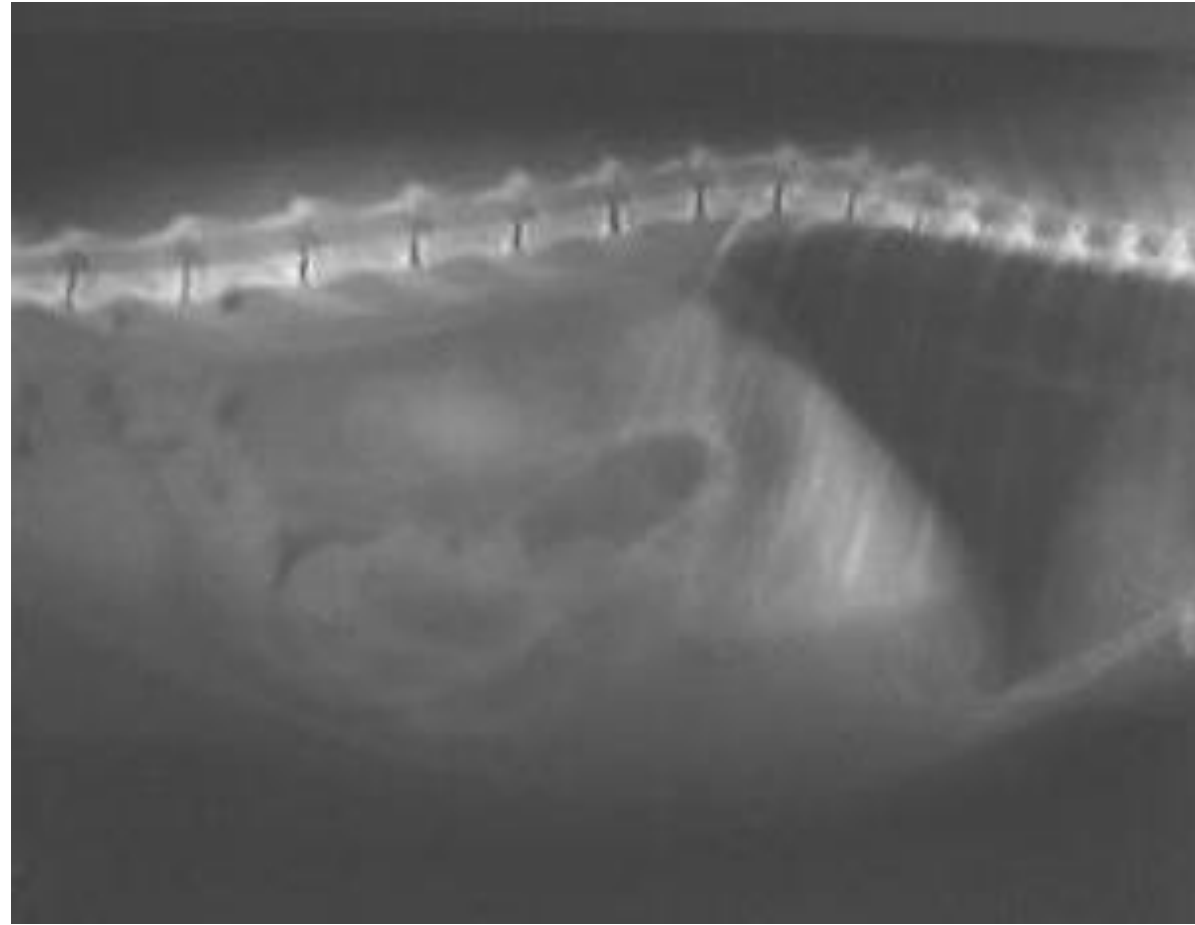
$$f(2) = 2$$

$$f(3) = f(4) = f(5) = f(6) = 0$$

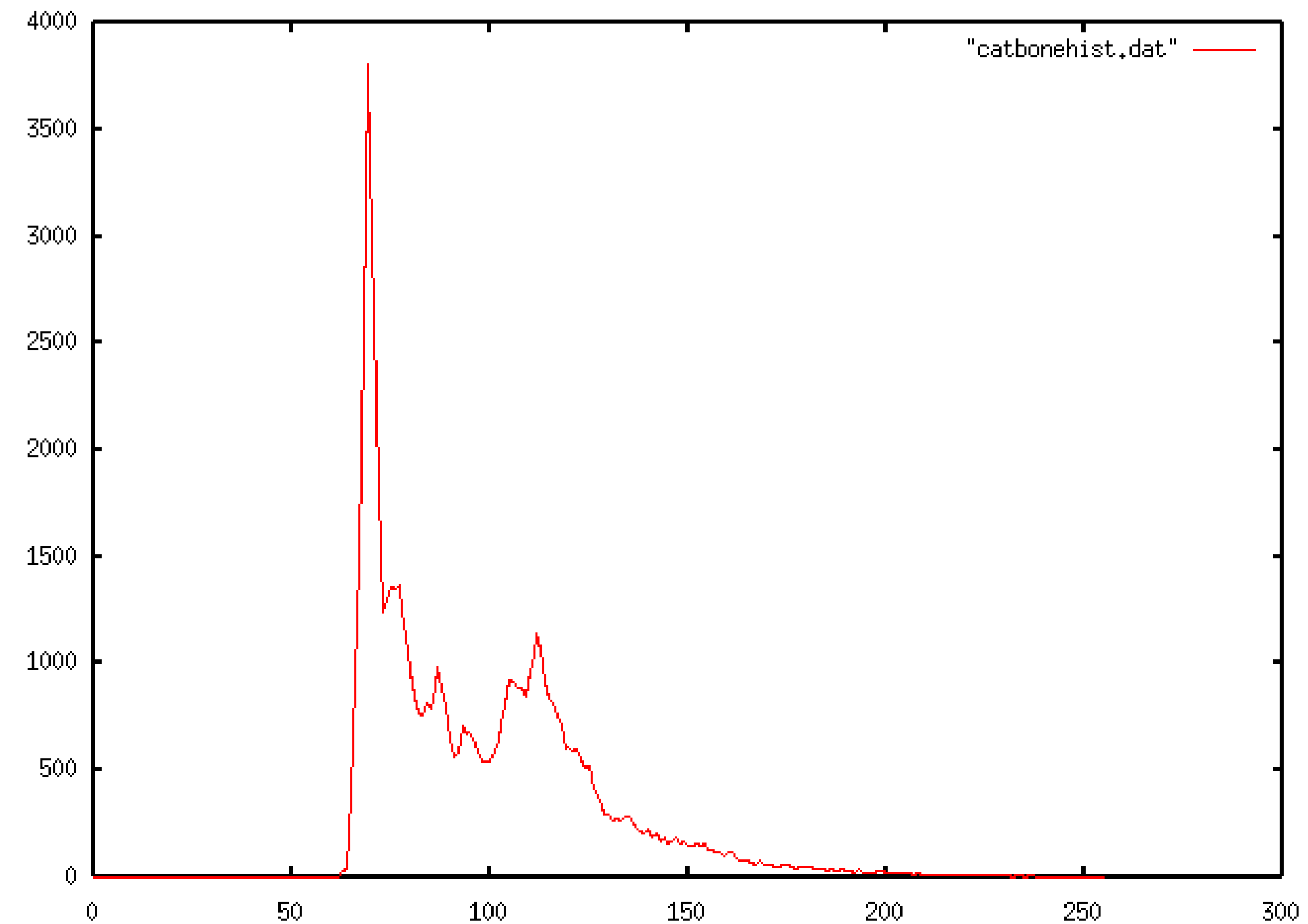
$$f(7) = 10$$



# Image Processing: Histogram Equalization



Example:

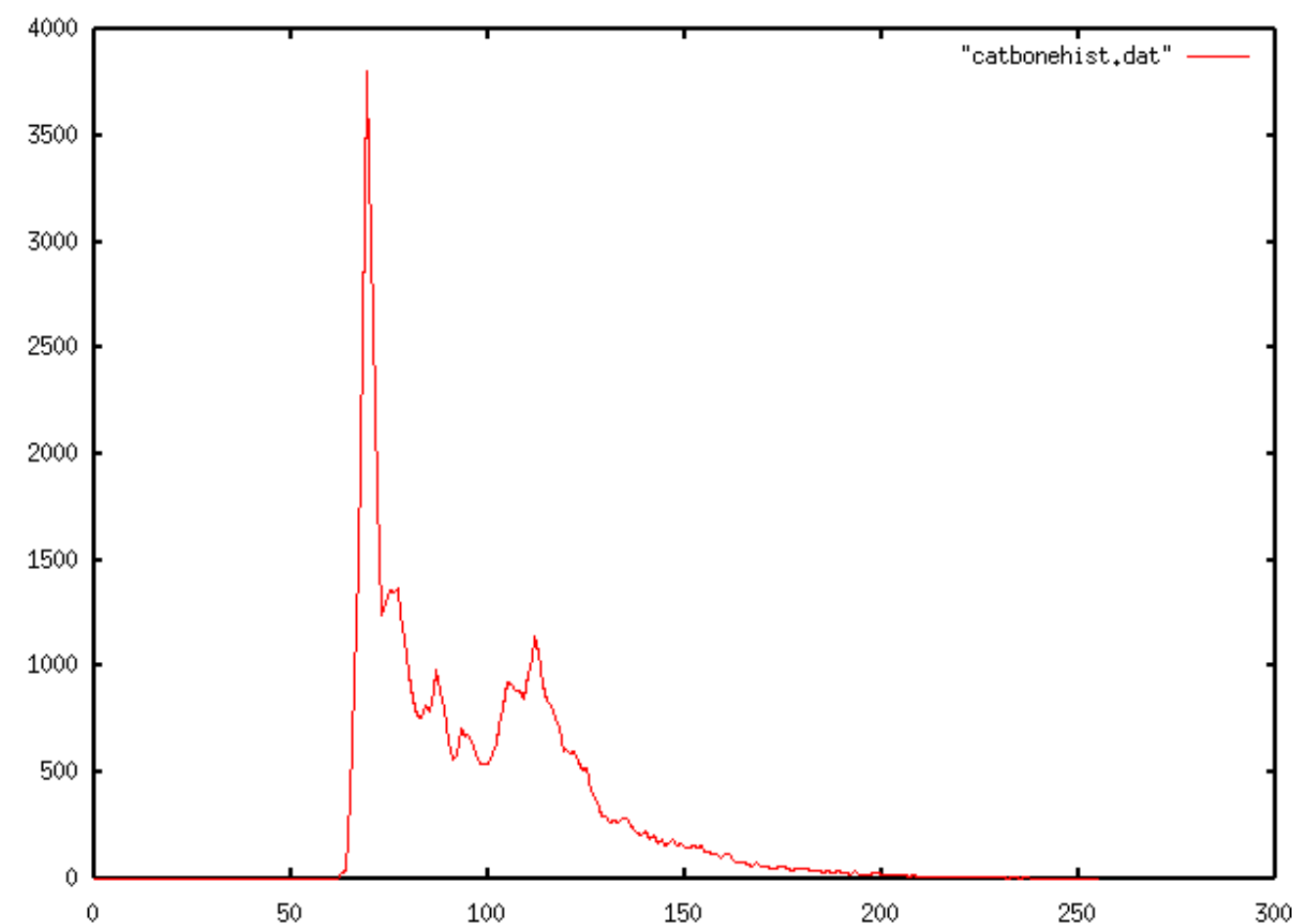
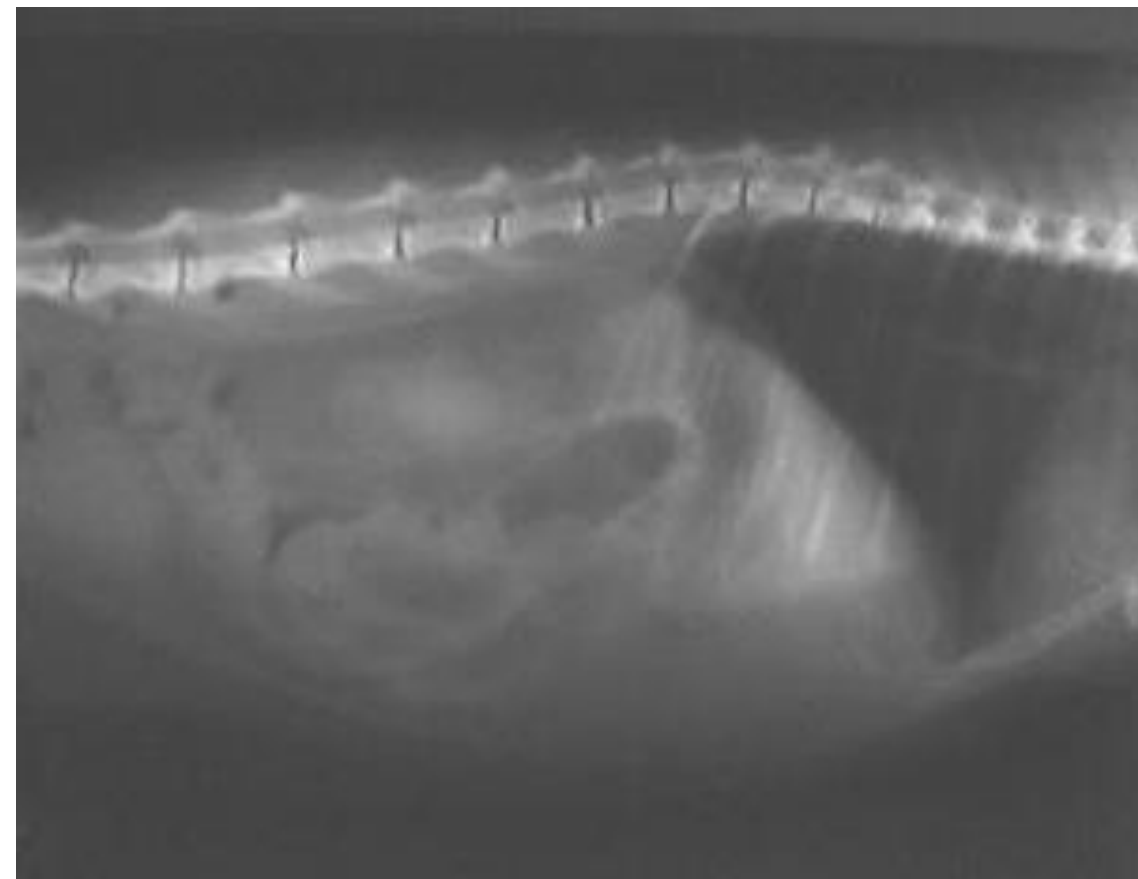


Histogram

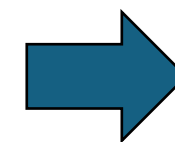
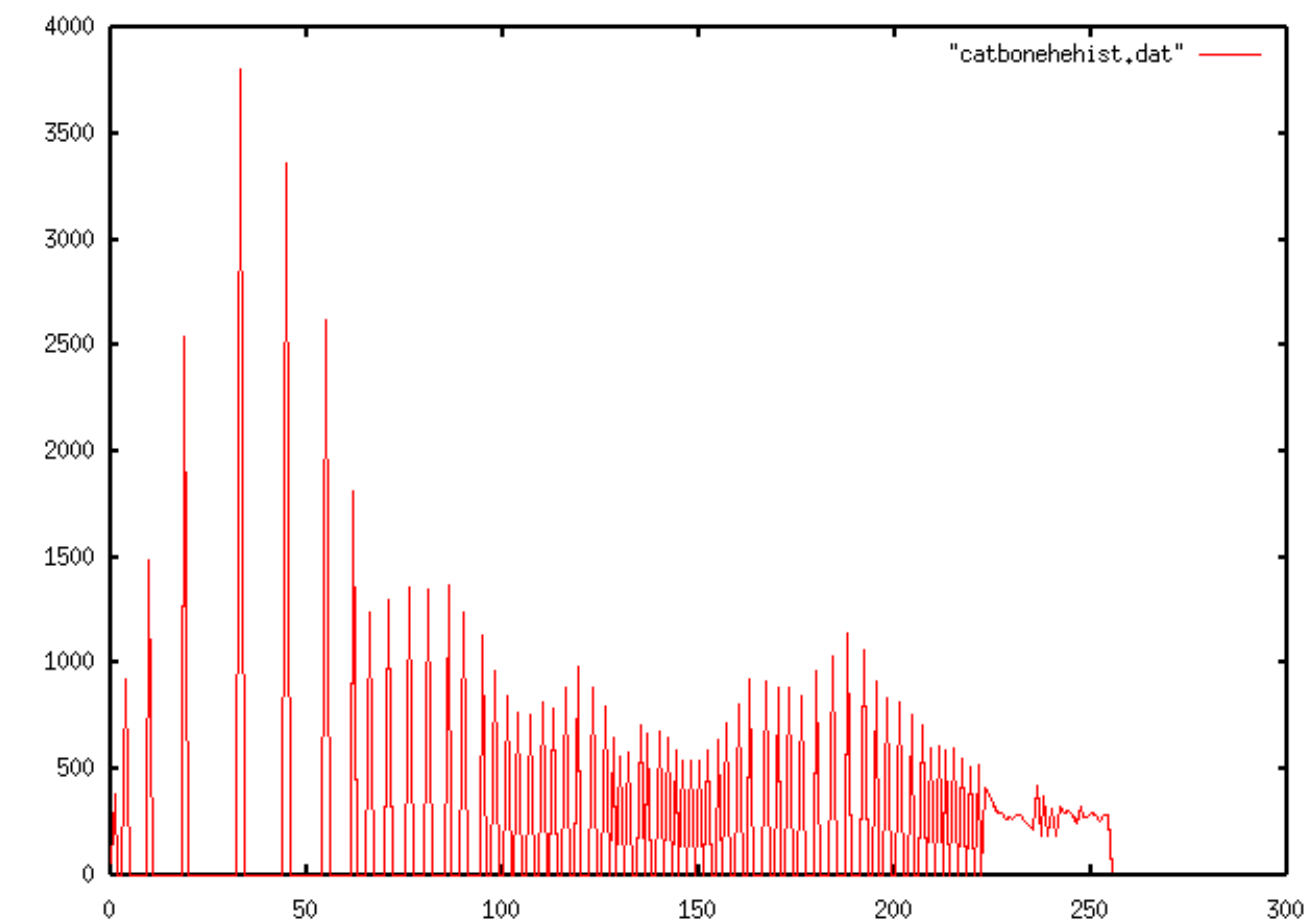
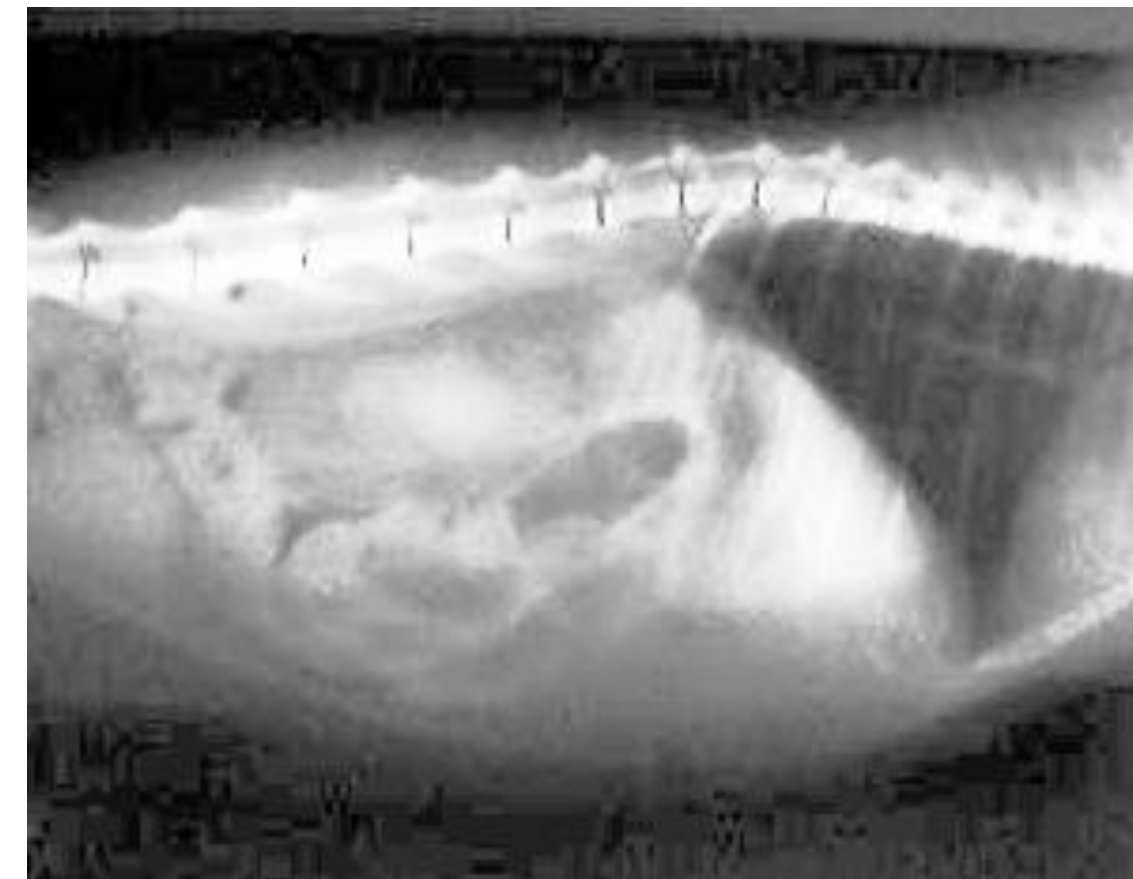
**Question:** Could two different images have exactly the same histogram?

# Histogram Equalization Improves Contrast

Low contrast



High contrast





# Image Processing: Histogram Equalization

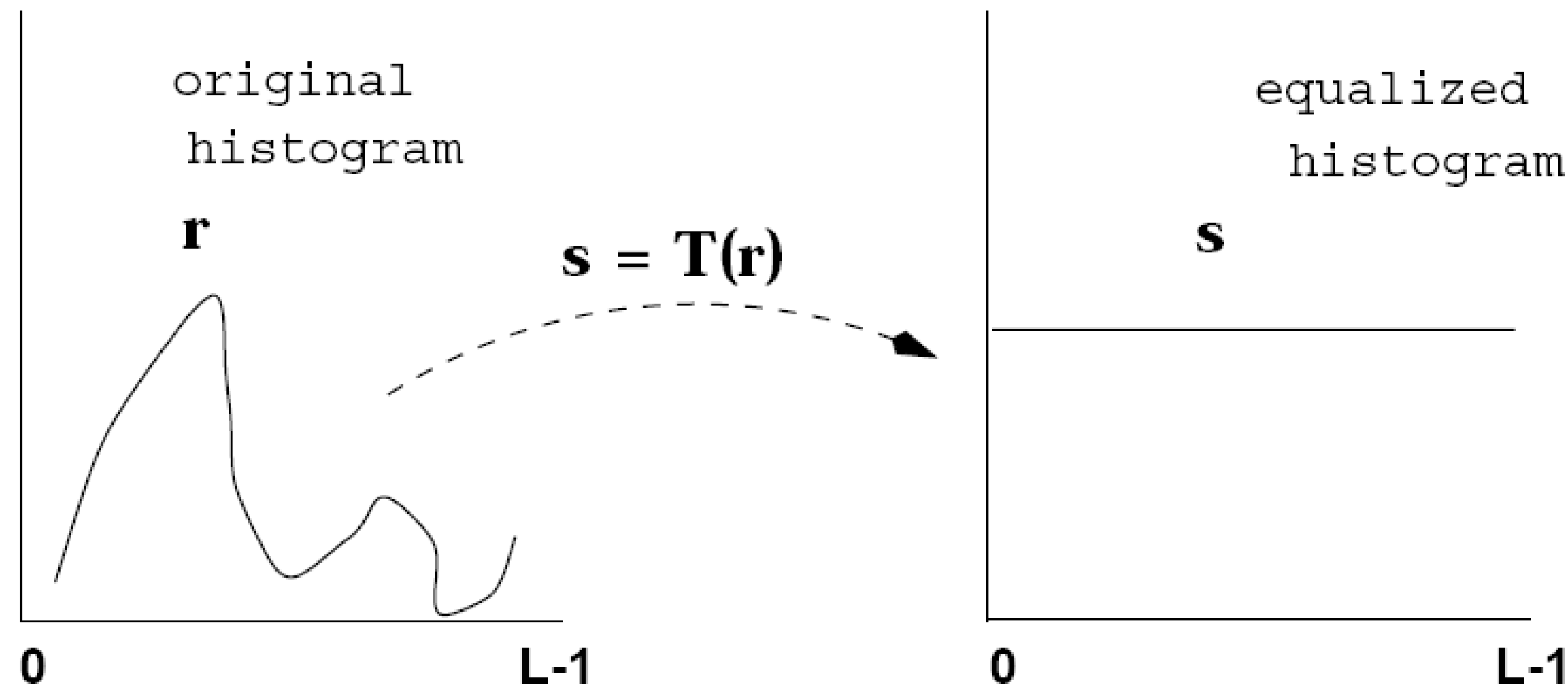
- To represent the **histogram as probabilities** (i.e., a normalized histogram), divide the frequency of each gray level by the total number of pixels.
- L is the total number of gray levels (typically 256),  $n_i$  is the number of pixels with value i, and n is the total number of pixels.

$$p_x(i) = \frac{n_i}{n}, 0 \leq i < L-1$$

$$P(0) = \frac{f(0)}{36} = \frac{1}{2}, P(1) = \frac{f(1)}{36} = \frac{1}{6}, P(2) = \frac{f(2)}{36} = \frac{1}{18}, P(3) = P(4) = P(5) = P(6) = 0, P(7) = \frac{f(7)}{36} = \frac{5}{18}$$

# Image Processing: Histogram Equalization

- In histogram equalization, the main idea is to **redistribute** the gray-level values uniformly.



This spreads contrast across the image instead of clustering values in a narrow band.  
 $P(0) = P(1) = P(2) = \dots = P(L-1)$

# Image Processing: Histogram Equalization

We would like to create a transformation  $T : [0, L - 1] \rightarrow [0, L - 1]$  to produce a new image  $Y$  from the input image  $X$ , with a flat histogram.

Such an image would have a linearized cumulative distribution function (CDF) across the value range.

**Cumulative Distribution Function (CDF)** of pixels in image  $X$  and  $Y$ :

$$cdf_X(i) = \sum_{j=0}^i p_X(j)$$

$$cdf_Y(i) = (i + 1)K, \text{ for } 0 \leq i < L \text{ for some constant } K.$$

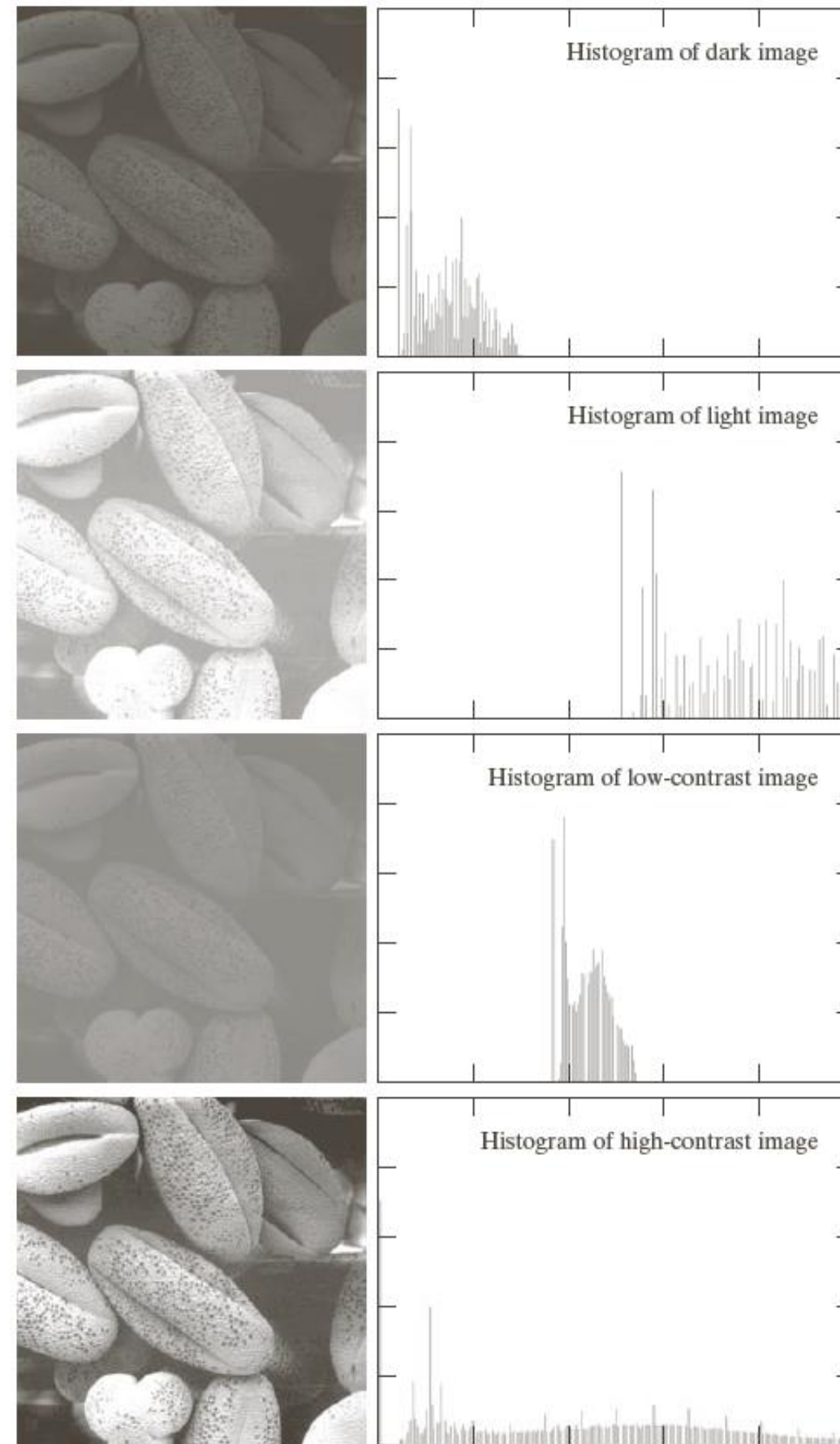
So, the transfer function can be defined as:

$$T(i) = \text{round}\left[\frac{cdf_X(i) - cdf_{\min}}{1 - cdf_{\min}} \times (L-1)\right] \xrightarrow{\text{if } cdf_{\min}=0} \text{round}[cdf_X(i) \times (L-1)]$$

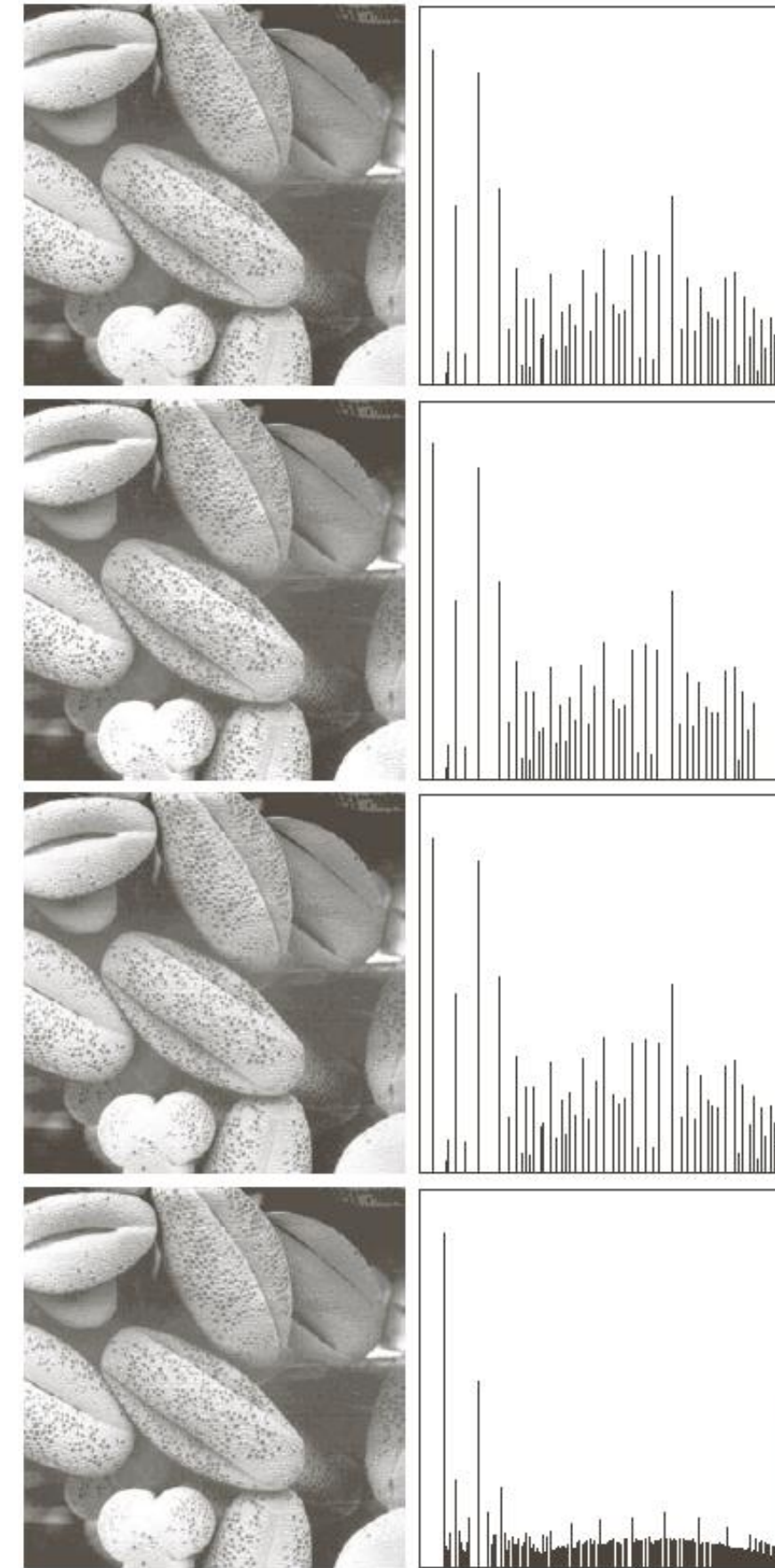


# Image Processing: Histogram Equalization

original images and histograms



equalized images and histograms



# Image Processing

- Image processing aims to enhance the information in an image that is most useful for later analysis.
- It is usually used to prepare images for further analysis and interpretation.
- An image can be represented as an  $n \times m$  array  $I(x,y) \rightarrow$  image intensity array.
- The cells are called pixels. Each number represents light intensity.
- We will discuss three important techniques: histogram equalization, **noise removal**, and edge detection.

# Image Processing: Noise Removal

- Real images always contain noise.
  - Typical noise sources include insufficient light, compression artifacts, digitization, electronic circuitry, and sensor faults.





# Image Processing: Noise Removal

- Real images always contain noise.
- **Smoothing** is a process used to reduce noise and sharp transitions in an image.
  - Smoothing tries to remove *isolated bright and dark regions* that are unlikely to represent a meaningful structure.
  - Smoothing filters work by making a pixel's intensity closer to the **average** or median intensity of its surroundings.
- Has side-effect of blurring image.

# Noise Removal: Averaging Filter

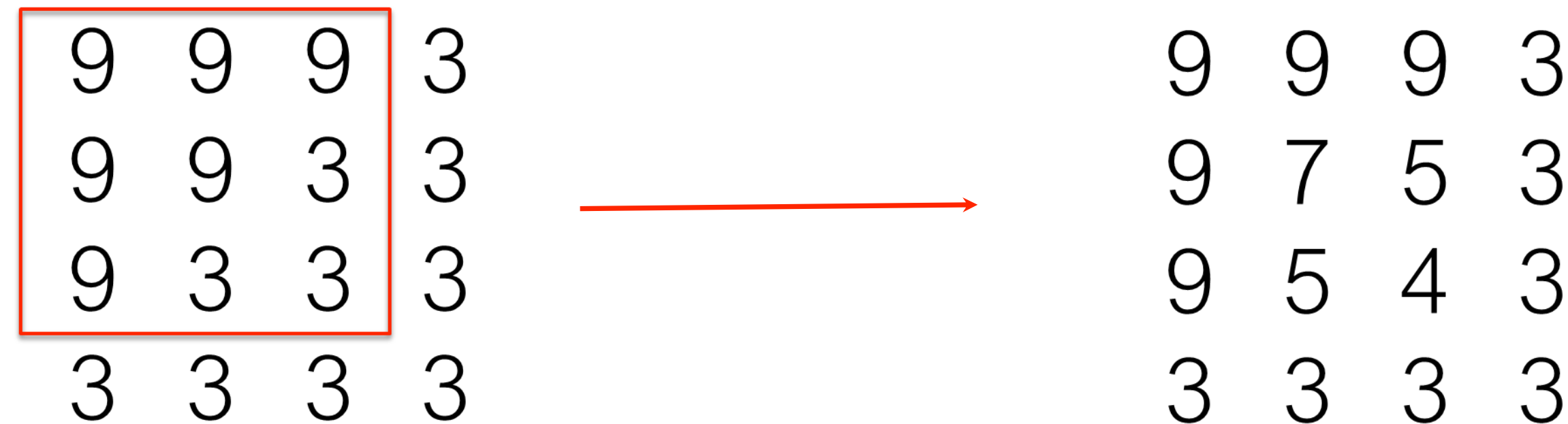
- Given a simple 4 x 4 picture matrix:

|   |   |   |   |
|---|---|---|---|
| 9 | 9 | 9 | 3 |
| 9 | 9 | 3 | 3 |
| 9 | 3 | 3 | 3 |
| 3 | 3 | 3 | 3 |

- Smooth this matrix using an averaging technique and a 3 x 3 pixel window.

# Noise Removal: Averaging Filter

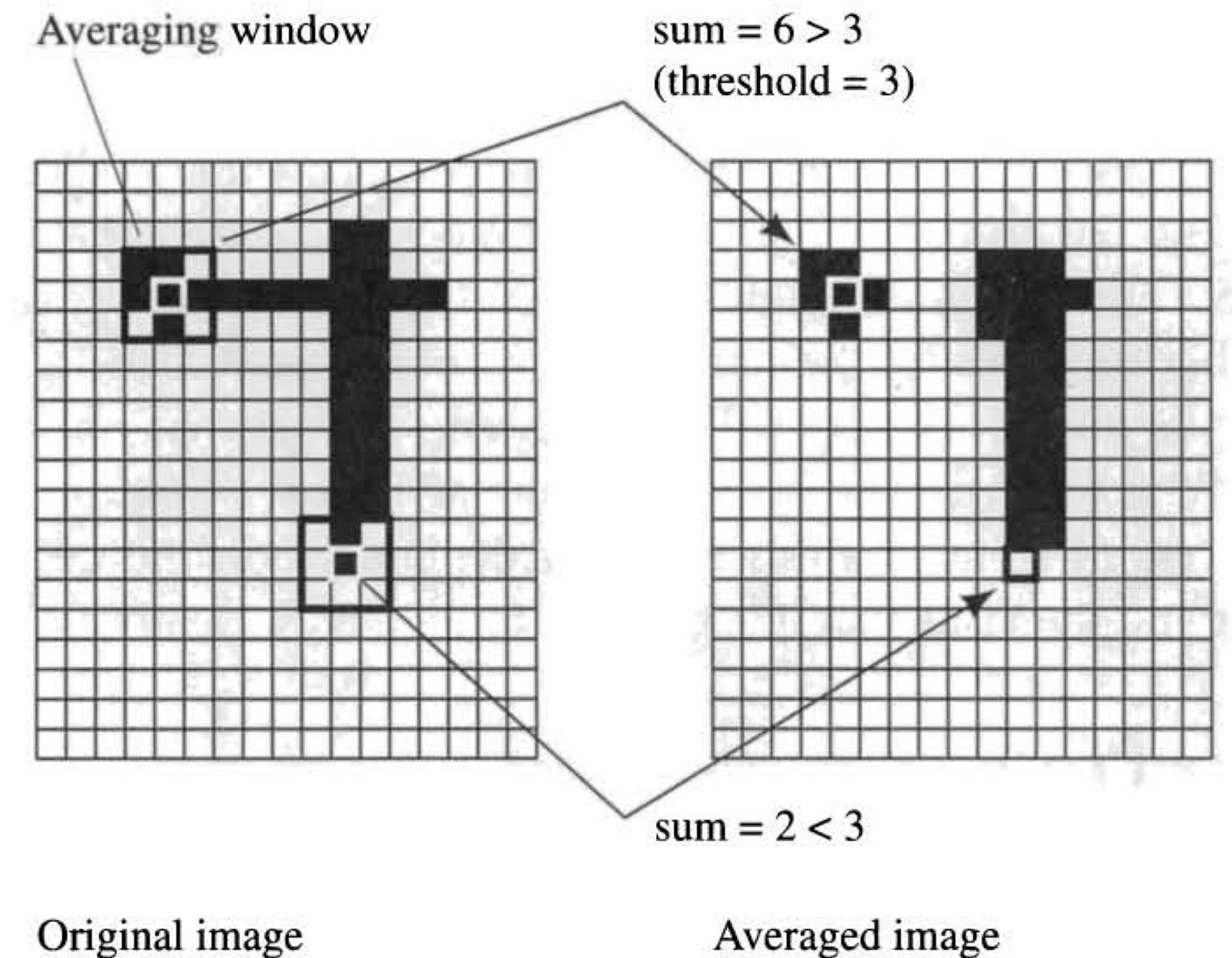
- There are four 3 x 3 pixel windows in the matrix.
- Replace the center value in each window with the average of all values in that window (**mean filter**).





# How Do We Remove Noise with an Averaging Filter?

- It can use a threshold.
- Larger rectangles achieve more smoothing.
- Broad lines are thickened and thin lines eliminated
- In the example,  $\varepsilon = 3$ , i.e., 0 if  $\text{sum} \leq 3$ , 1 otherwise.

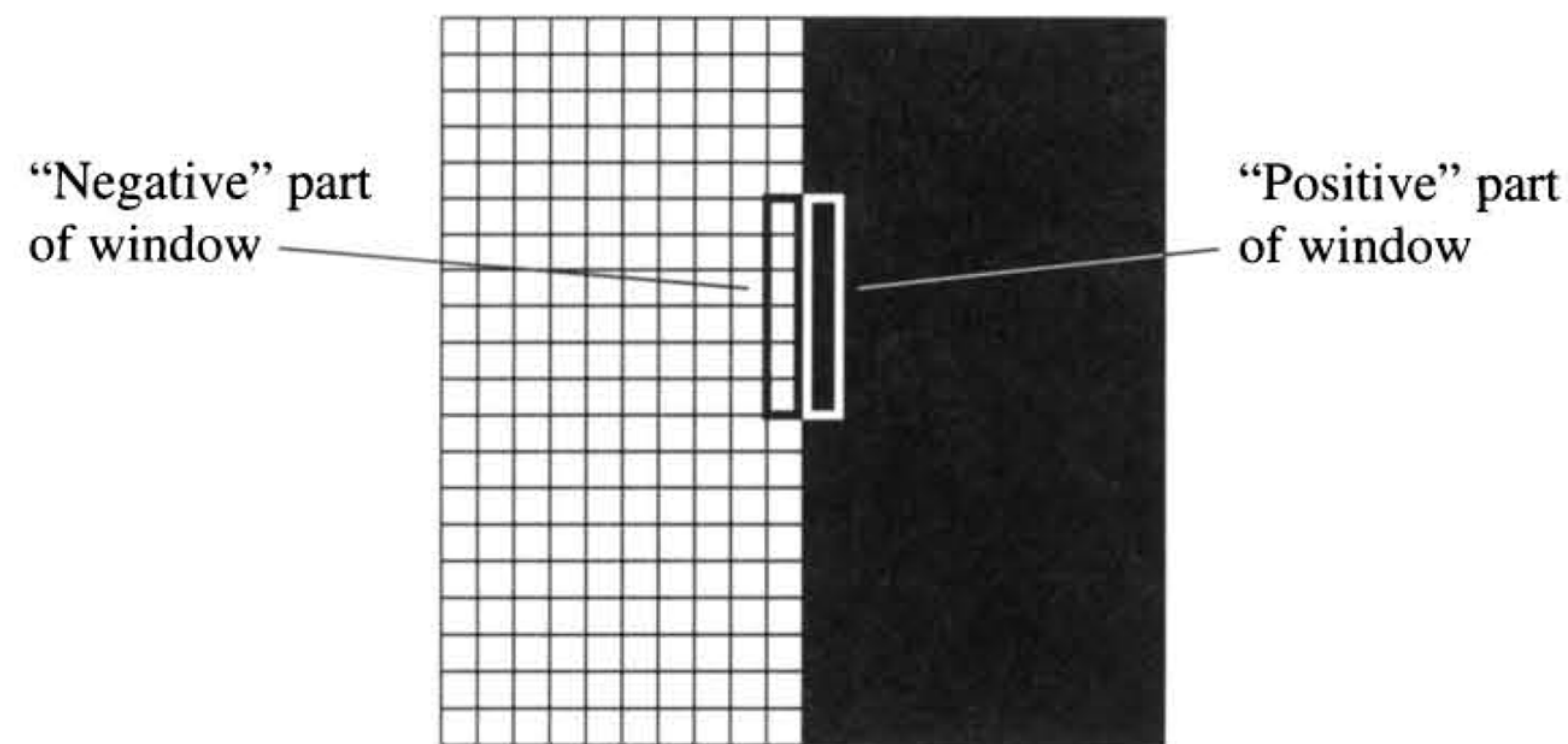


# Image Processing

- Image processing aims to enhance the information in an image that is most useful for later analysis.
- It is usually used to prepare images for further analysis and interpretation.
- An image can be represented as an  $n \times m$  array  $I(x,y) \rightarrow$  image intensity array.
- The cells are called pixels. Each number represents light intensity.
- We will discuss three important techniques: histogram equalization, noise removal, and **edge detection**.

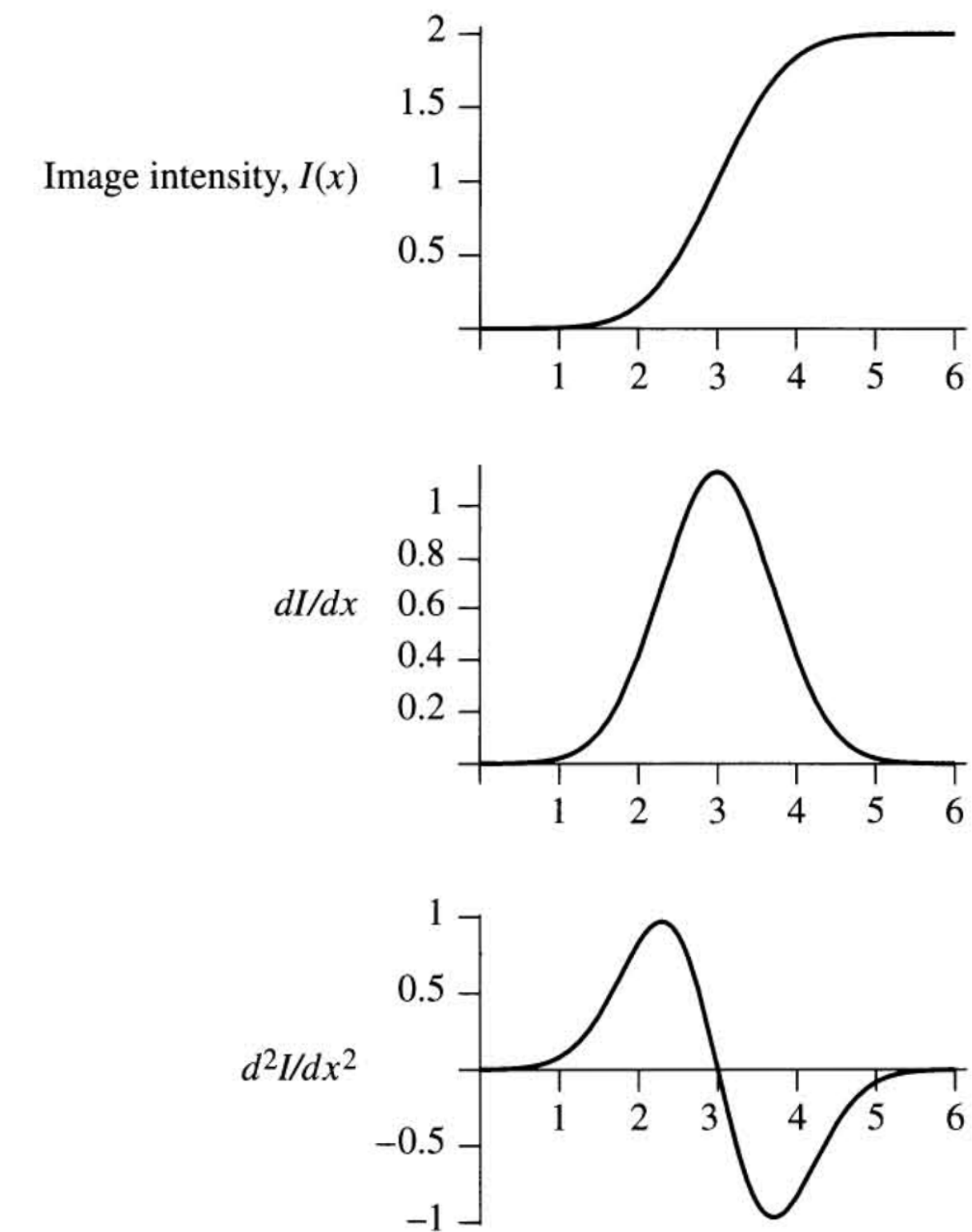
# Where Do Edges Come From?

- Edges are used to build a line drawing.
- Outlines can be compared with object models.
- Edges are parts of the image with markedly different property values (e.g., intensity)



**Figure 6.9**

Edge Enhancement



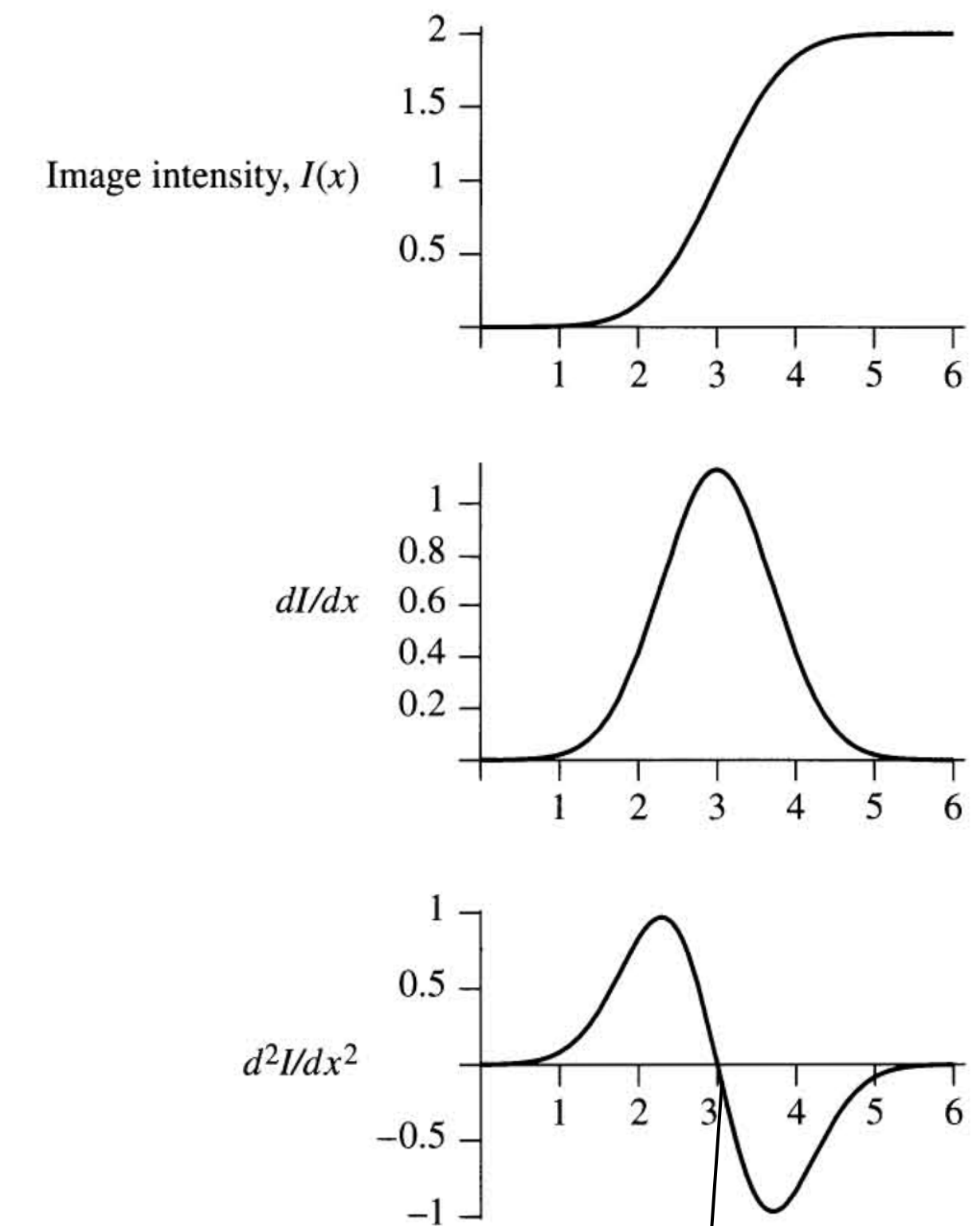
**Figure 6.10**

Taking Derivatives of Image Intensity



# Image Processing: Edge Detection

- ✓ An edge is a location in the image where intensity changes sharply.
- ✓ **First derivative:** measures the *rate of change*:
  - Small in flat regions.
  - Edges often correspond to peaks or valleys in the first derivative.
- ✓ **Second derivative:**
  - At an edge we have a min/max in our first derivative → cause 0 in second derivative.



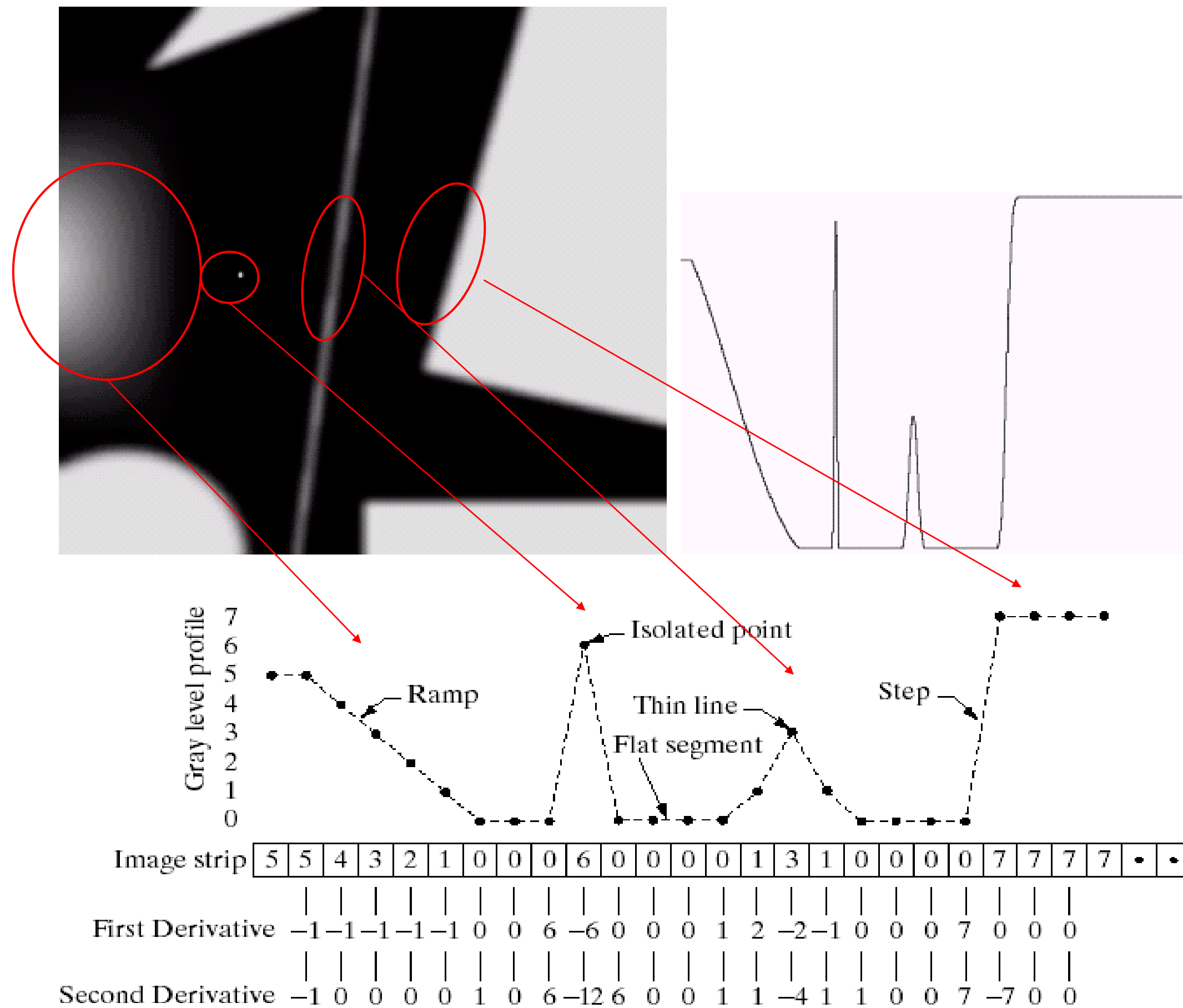
**Figure 6.10**

Taking Derivatives of Image Intensity

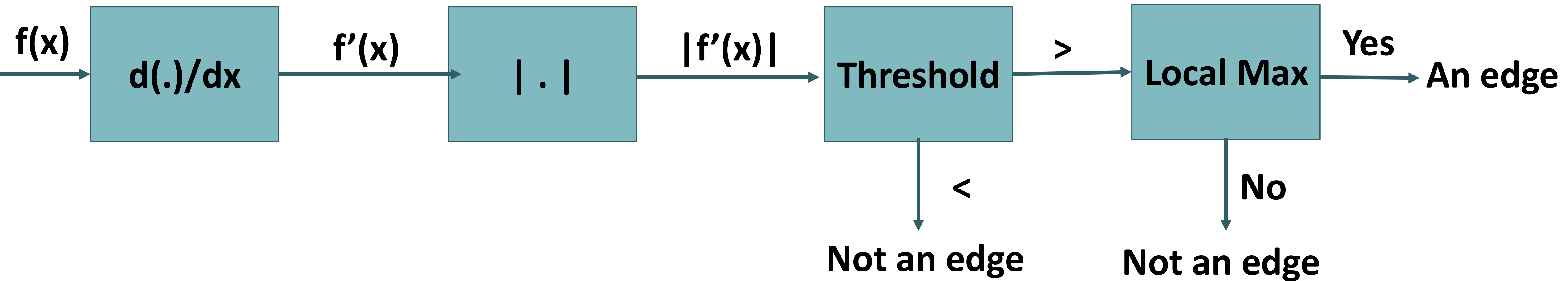
**zero-crossing**

# Image Processing: Edge Detection

Horizontal intensity profile through the center of the image, including the isolated noise point.



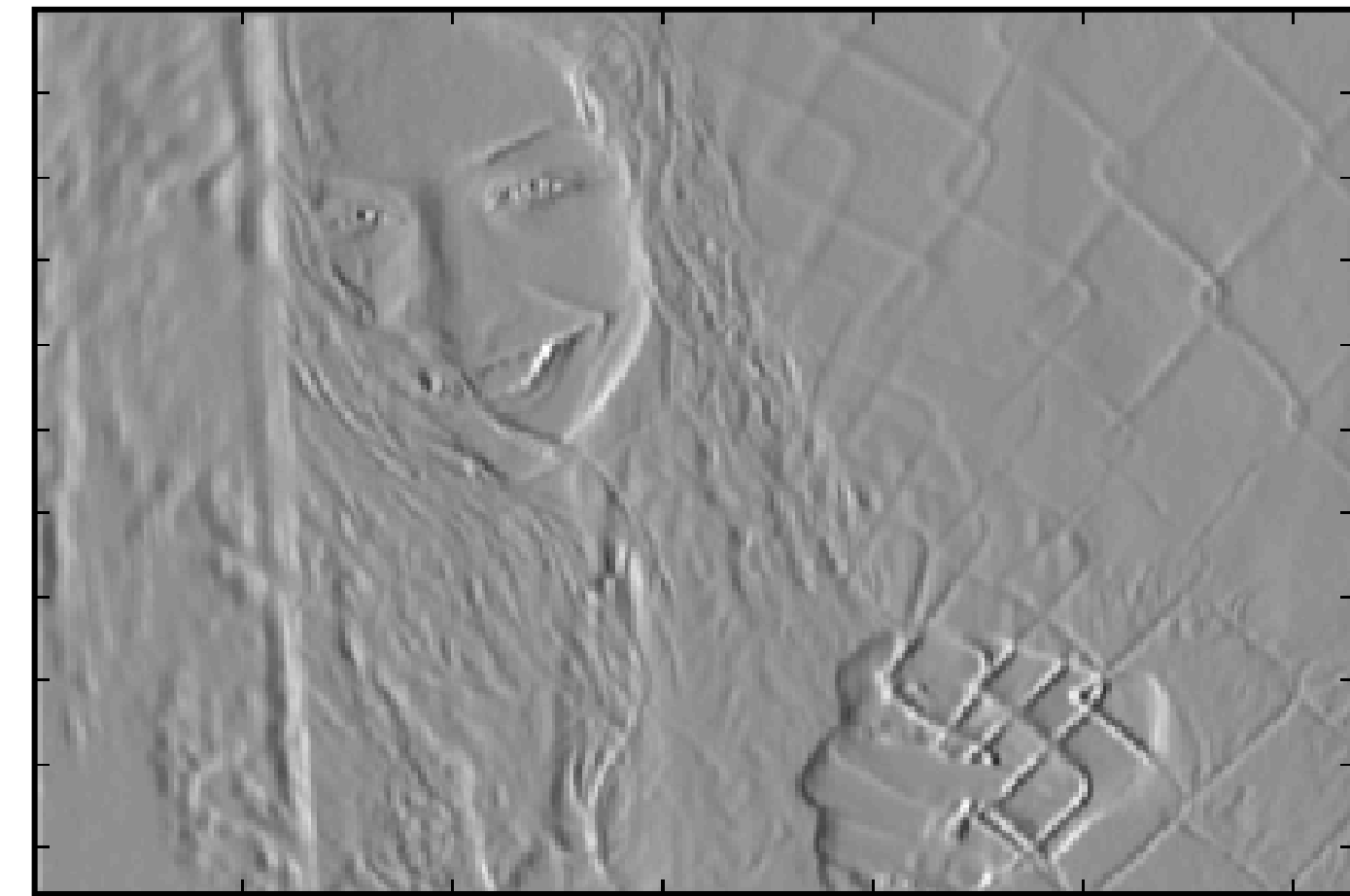
# Image Processing: Edge Detection



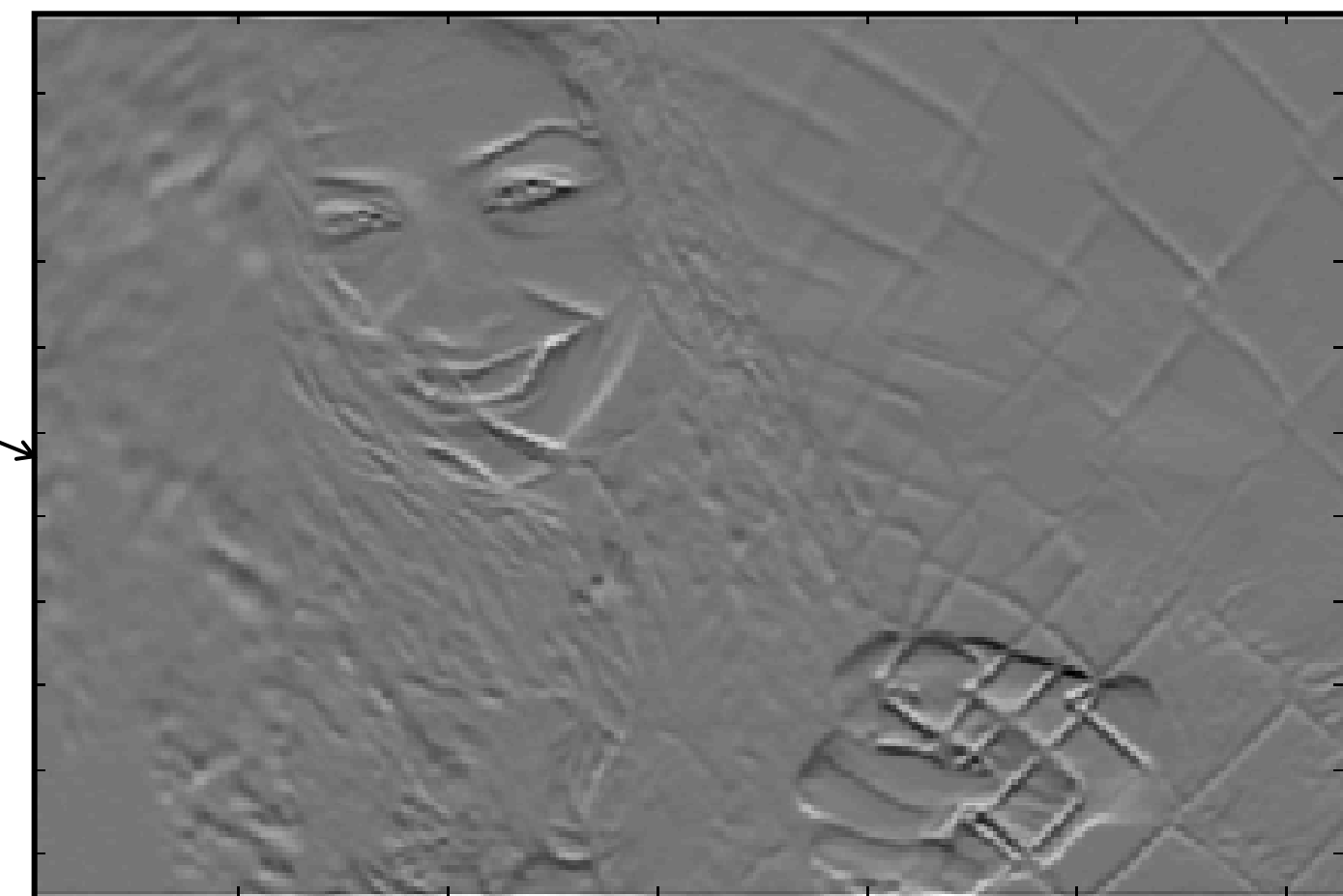


# Image Processing: Edge Detection

- How can we calculate the derivative of an image?



$$\frac{\partial f}{\partial x}$$



$$\frac{\partial f}{\partial y}$$

# Image Processing: Edge Detection

- The **Laplacian filter** is a spatial filter primarily used for edge detection.
- It computes second derivatives of the image, highlighting locations where pixel intensity changes rapidly.

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

From Taylor expansion we will have:

$$\frac{\partial^2 f}{\partial x^2} \approx f(x+1, y) - 2f(x, y) + f(x-1, y)$$

$$\frac{\partial^2 f}{\partial y^2} \approx f(x, y+1) - 2f(x, y) + f(x, y-1)$$

# Image Processing: Edge Detection

$$\nabla^2 f(x, y) \approx f(x + 1, y) + f(x - 1, y) + f(x, y + 1) + f(x, y - 1) - 4f(x, y)$$

- We can write the kernel as follow:

$$\begin{pmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{pmatrix}$$

- There are two common Laplacian filter:

Less sensitive

|    |    |    |
|----|----|----|
| 0  | -1 | 0  |
| -1 | 4  | -1 |
| 0  | -1 | 0  |

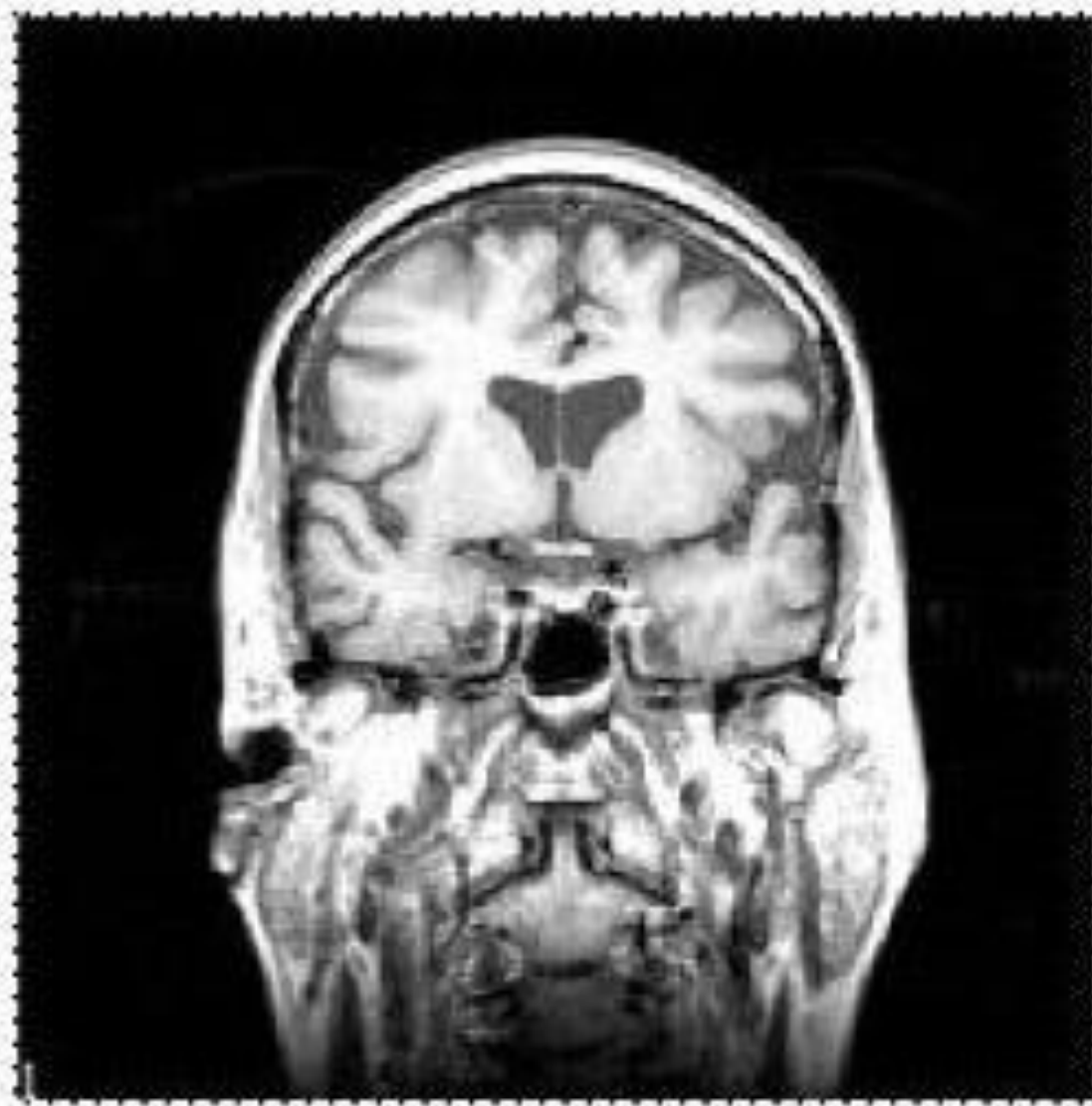
|    |    |    |
|----|----|----|
| -1 | -1 | -1 |
| -1 | 8  | -1 |
| -1 | -1 | -1 |

More sensitive, detects more edges

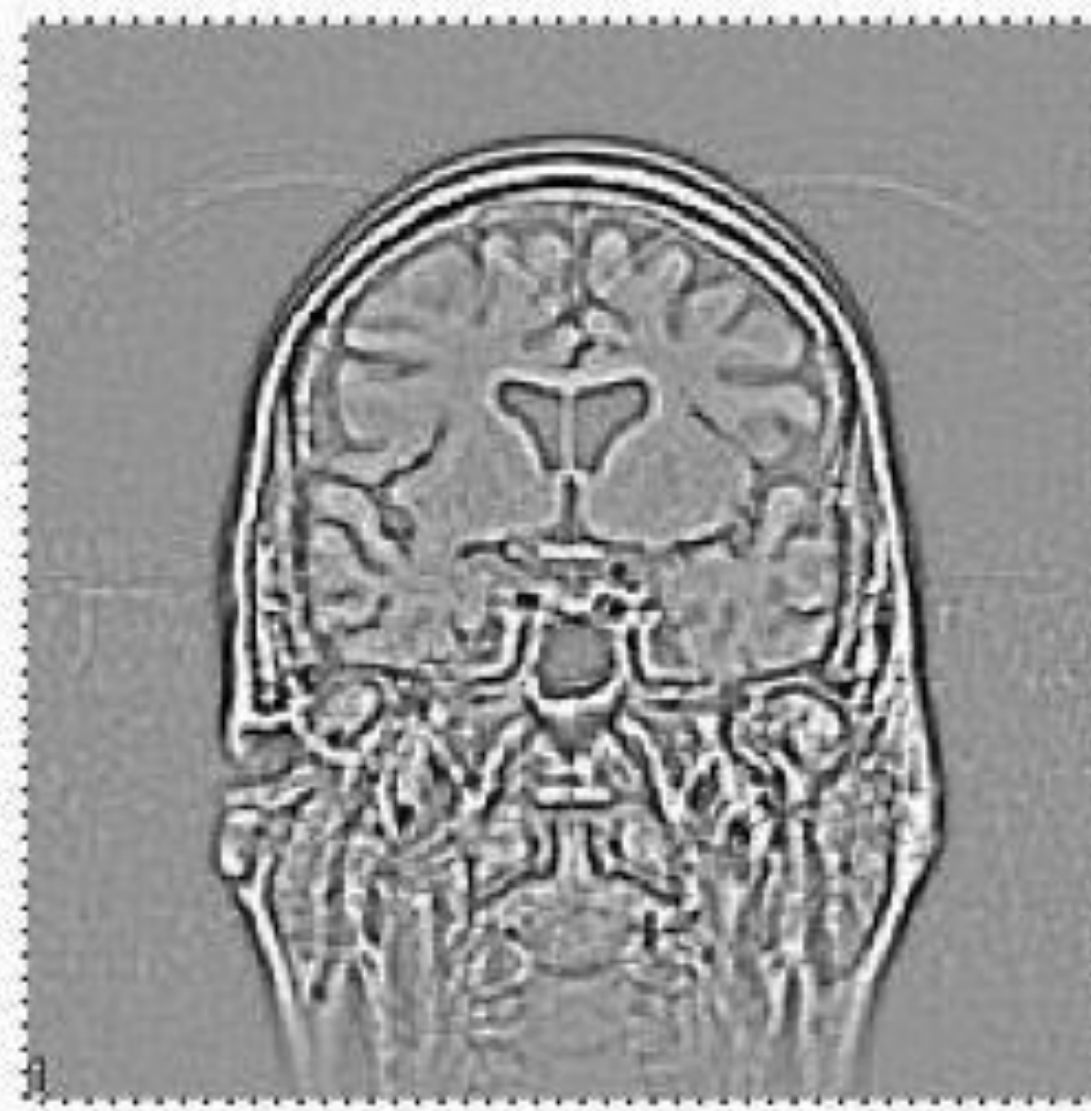


# Why Canny Works Better in Practice

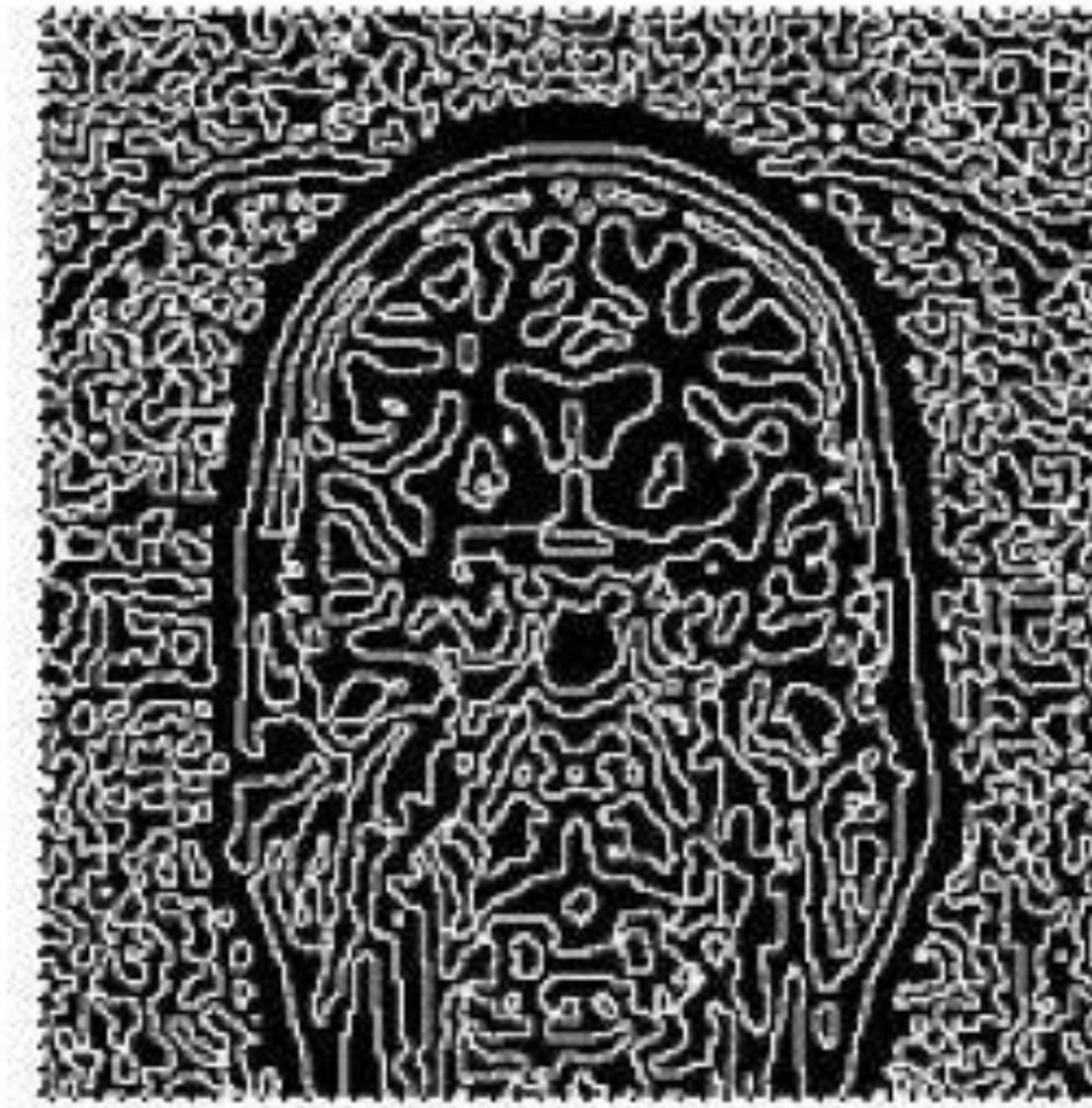
- Averaging and edge enhancement can be combined.
- For instance, using a Laplacian filter.



(A) Original MR image



(B) Laplacian results



(C) Extraction of the zero crossing of the Laplacian (object edges)

# Image Processing: Edge Detection

The **Canny edge detector** can be broken down into four main steps:

1. Apply **Gaussian filter** to smooth the image and reduce noise.
2. Compute the intensity gradient magnitude and direction at each pixel.
3. Apply **gradient magnitude thresholding** and suppress non-maximum responses.
4. Apply double thresholding to classify which edges are **strong**, **weak**, or **non-edges**.



# Image Processing: Edge Detection

1. Apply **Gaussian filter** to smooth the image in order to reduce noise

- **Gaussian filter** is a **filter** that gives **more weight to pixels near the center** of the window and **less to those farther away** (instead of averaging all pixels equally as in a mean filter).
- The 2D Gaussian function is:

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

where  $(x, y)$  is pixel coordinates relative to the center of the filter, and  $\sigma$  is the standard deviation.

**Example (3×3 Gaussian kernel,  $\sigma \approx 1$ )**

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$



# Image Processing: Edge Detection

2. Find the intensity gradients of the image and direction of each pixel.

$$\begin{cases} \frac{\partial f}{\partial x} = G_x : \text{Gradient in the x-direction (vertical edges)} \\ \frac{\partial f}{\partial y} = G_y : \text{Gradient in the y-direction (horizontal edges)} \end{cases}$$

Edges in an image are places where **intensity changes rapidly** — that means **high derivatives**.

- **Horizontal edges:** Large values in  $G_y$
- **Vertical edges :** Large values in  $G_x$

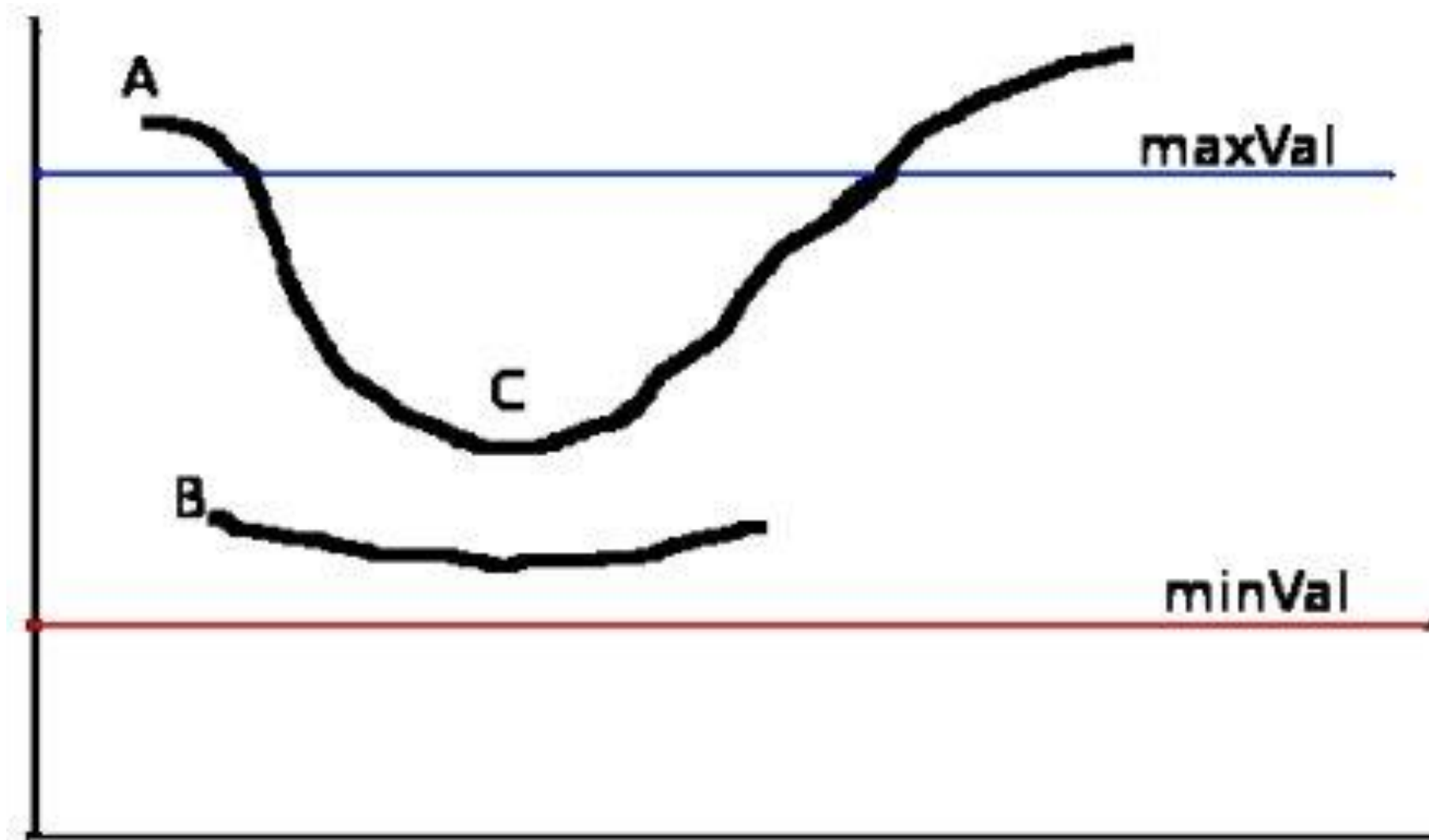
Then we compute the **magnitude and direction of the gradient** at each pixel:

- Gradient Magnitude=  $\sqrt{G_x^2 + G_y^2}$
- Gradient Direction=  $\arctan(G_y/G_x)$

# Image Processing: Edge Detection

3. Apply **gradient magnitude thresholding** and suppress non-maximum responses.

- Any edges with intensity gradient more than  $\text{maxVal}$  are treated as strong edges.
- Those below  $\text{minVal}$  are treated as non-edges and discarded.
- Pixels between the two thresholds are kept only if they are connected to strong-edge pixels.



[https://docs.opencv.org/4.x/da/d22/tutorial\\_py\\_canny.html](https://docs.opencv.org/4.x/da/d22/tutorial_py_canny.html)

# Roadmap: From Classical Vision to CNNs

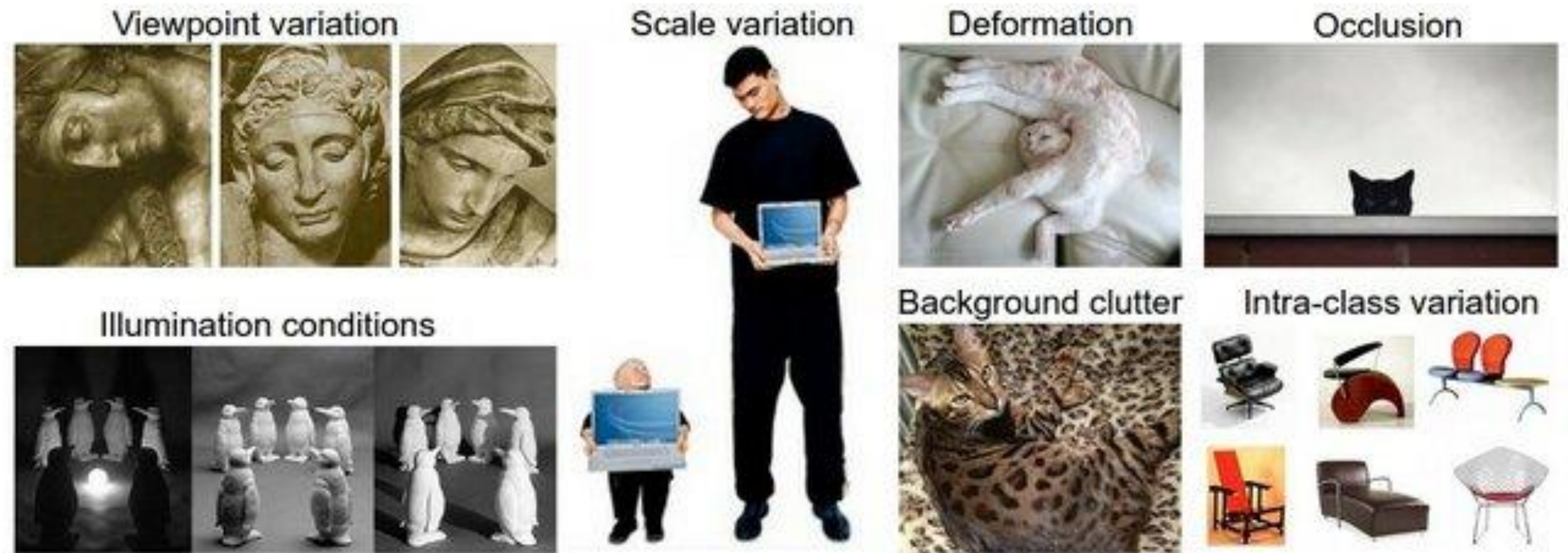
- Introduction
- Image Processing
  - Histogram Equalization
  - Noise Removal
  - Edge Detection
- **Computer Vision**
  - Convolutional Neural Networks
    - ✓ VGG16
    - ✓ AlexNet
- Cognitive Vision



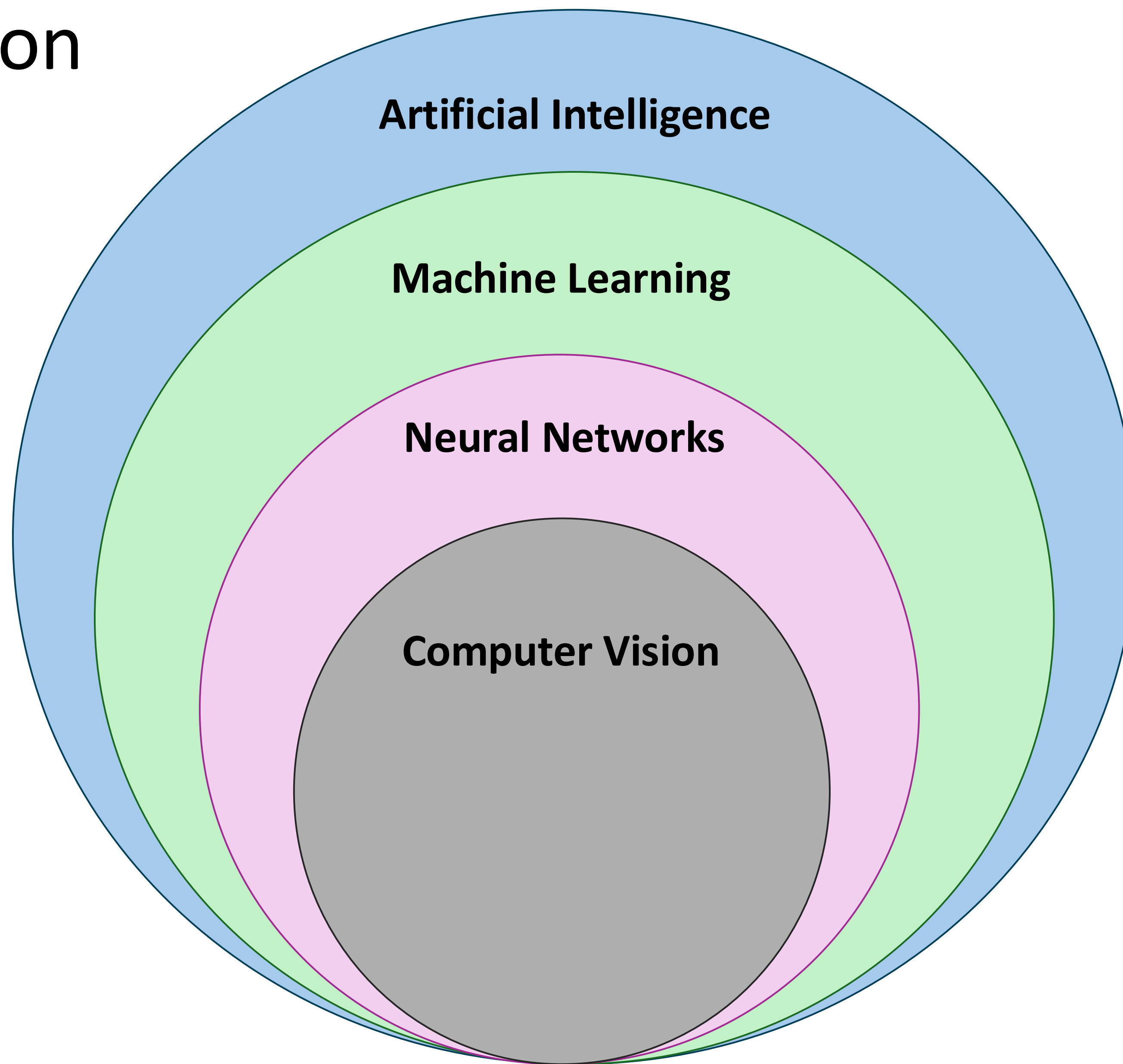
# Computer Vision

## Challenges in CV

- Variations in viewpoint
- Illumination changes
- Occlusion and deformation
- Background clutter



# Computer Vision



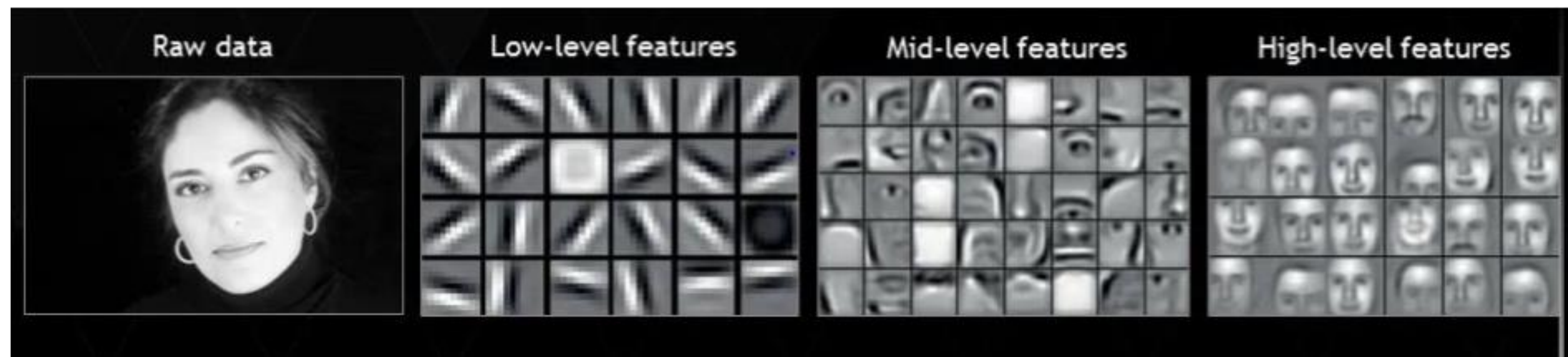
A comparative view of how AI, machine learning, neural networks, and computer vision relate to one another.



# Computer Vision: CNN

## Convolutions + Neural Networks = Convolutional Neural Networks (CNNs)

- **Convolutional neural networks (CNNs)** are a class of neural networks widely used in computer vision tasks.
- A CNN is composed of building blocks such as convolution layers, pooling layers, and fully connected layers, and it learns spatial hierarchies of features automatically.



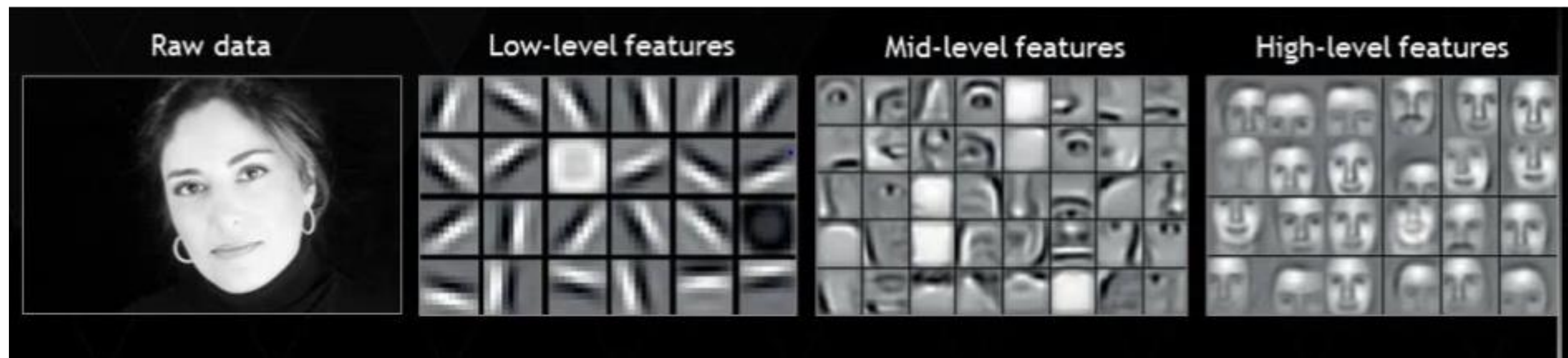
<https://www.analyticsvidhya.com/blog/2017/04/comparison-between-deep-learning-machine-learning/>



# Computer Vision: CNN

## Convolutions + Neural Networks = Convolutional Neural Networks (CNNs)

- CNNs will try to learn **low-level features** such as edges and lines in early layers, then object parts, and finally higher-level object representations in deeper layers.
- CNNs can be viewed as gradually transforming images into representations in which the classes become easier to separate.

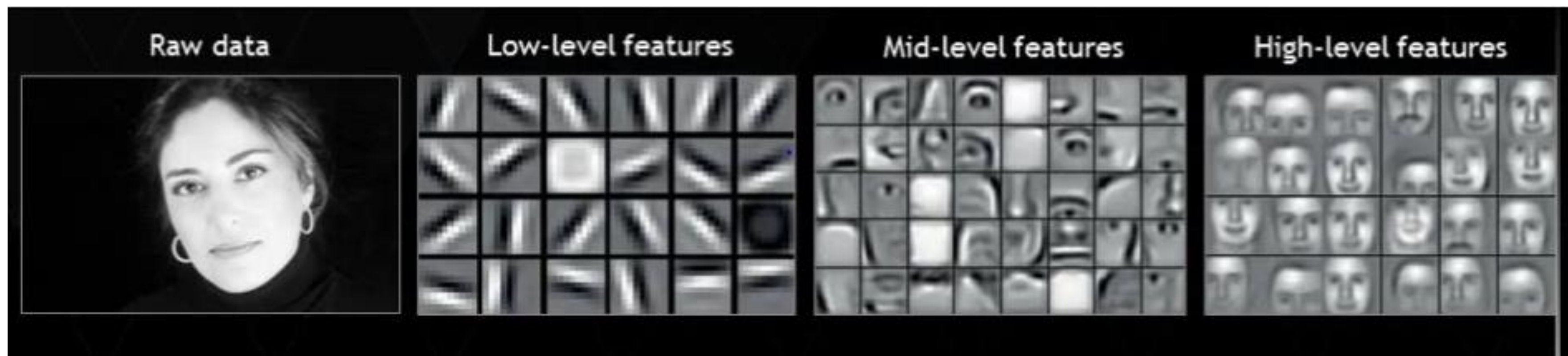


<https://www.analyticsvidhya.com/blog/2017/04/comparison-between-deep-learning-machine-learning/>

# Computer Vision: CNN

## Convolutions + Neural Networks = Convolutional Neural Networks (CNNs)

- CNNs are used for image recognition, object detection, and classification tasks.
- CNNs excel at feature extraction, learning complex and abstract features from input data.
- **Parameter sharing** in CNNs reduces both computational cost and memory requirements, which makes them efficient.



<https://www.analyticsvidhya.com/blog/2017/04/comparison-between-deep-learning-machine-learning/>

# Computer Vision: CNN

Convolution is a mathematical operator.

## Continuous convolution

$$s(t) = (x * w)(t) = \int x(a)w(t - a) da$$

## Discrete convolution

$$s(t) = (x * w)(t) = \sum_{a=-\infty}^{\infty} x(a)w(t - a)$$

## Two-dimensional convolution

$$S(j, k) = (K * I)(j, k) = \sum_{m=-\infty}^{\infty} \sum_{n=-\infty}^{\infty} K(m, n) I(j-m, k-n)$$



# Computer Vision: CNN

**Two-dimensional convolution:**

$$S(j, k) = (K * I)(j, k) = \sum_{m=-\infty}^{\infty} \sum_{n=-\infty}^{\infty} K(m, n) I(j-m, k-n)$$

**For image I and filter K:**

$$S(j, k) = (K * I)(j, k) = \sum_{m=-a}^a \sum_{n=-b}^b K(m, n) I(j-m, k-n)$$

Where  $(2a+1) \times (2b+1)$  is the size of the filter (kernel).

# Computer Vision: CNN

## Core CNN Components:

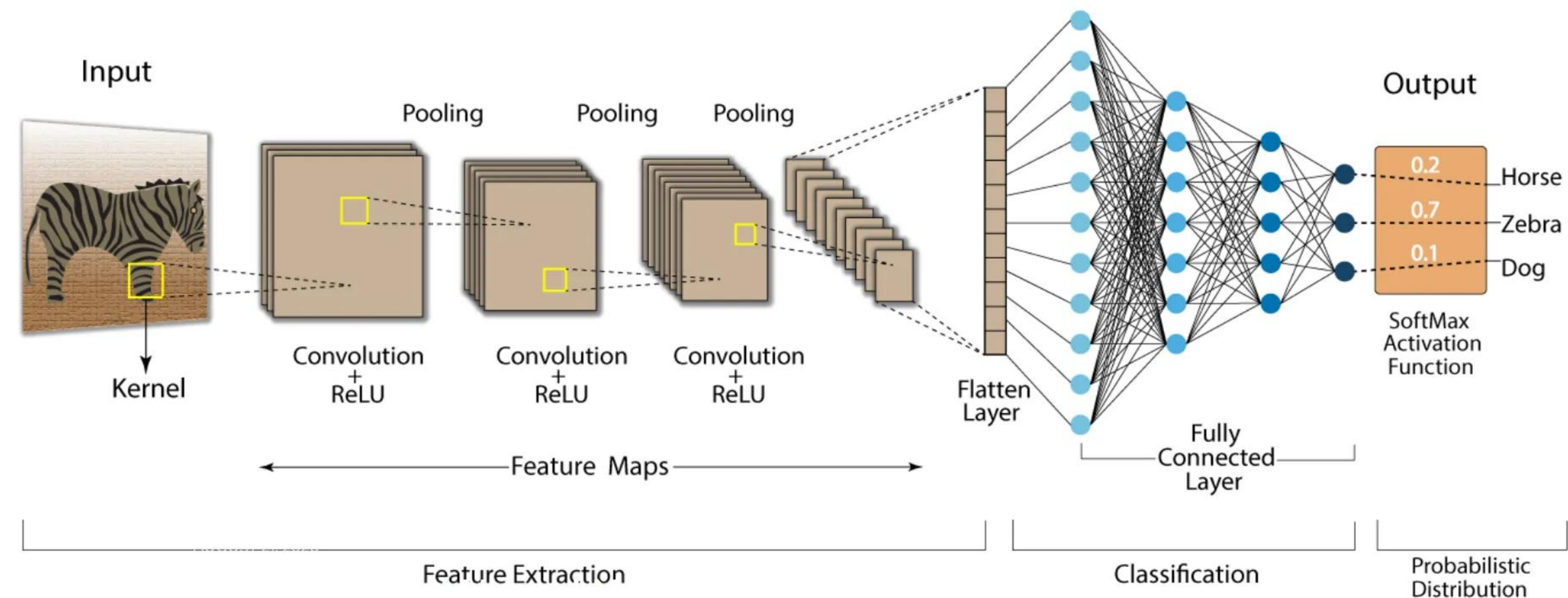
**1. Convolutional Layers** – Apply filters (kernels) to detect features such as edges, textures, and patterns.

**2. Pooling Layers** – Reduce spatial dimensions while retaining important features.

**3. Flattening layer** – convert feature maps into a 1D vector

**4. Fully Connected Layers** – Connect extracted features to the final prediction stage.

**5. Output layer** – produce the final class scores or probabilities.

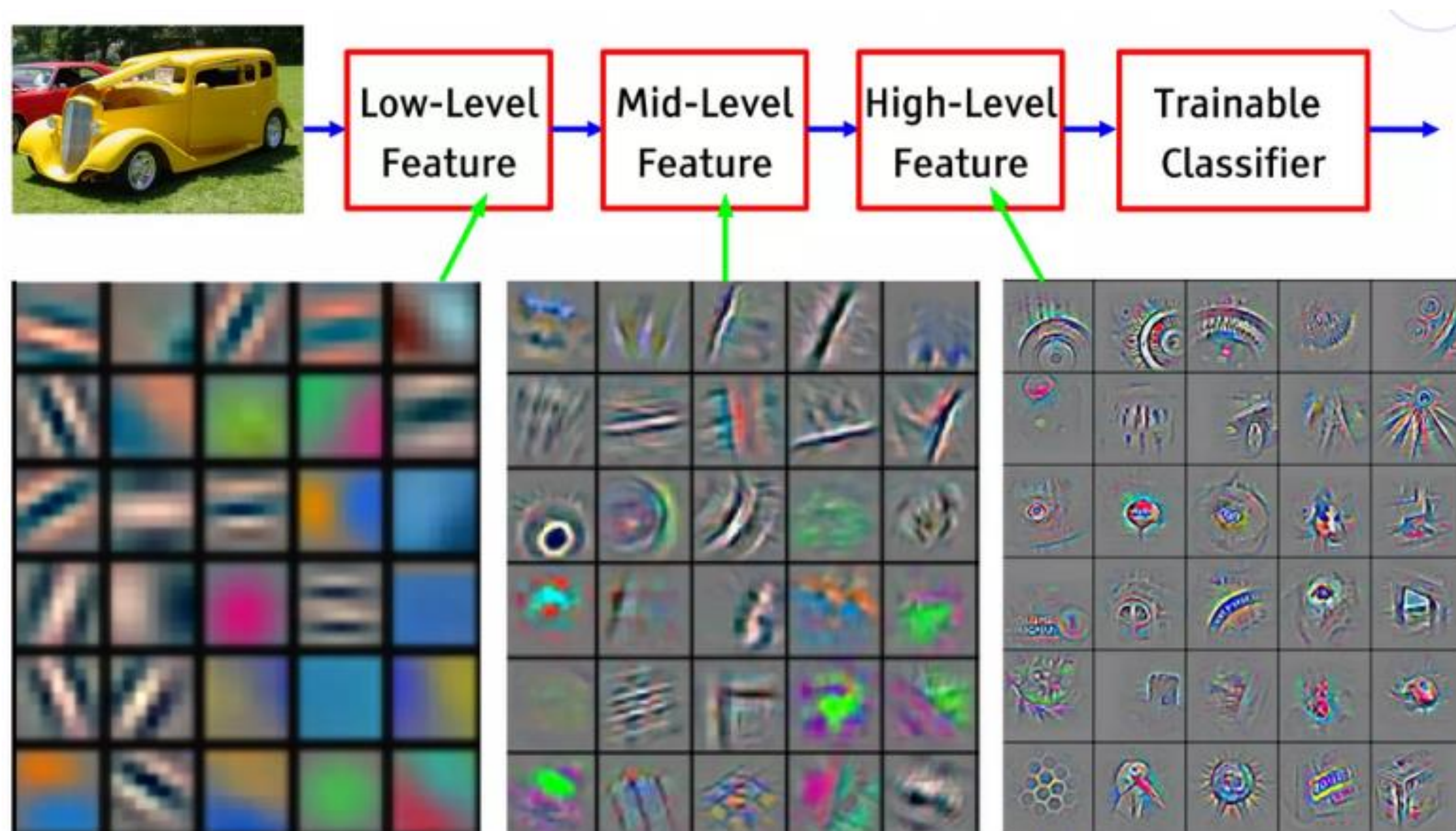


Sourced from [nafizshahriar.medium.com](https://nafizshahriar.medium.com) on convolutional neural networks and deep learning.



# Computer Vision: CNN

- Convolution helps us extract important features from an image, such as edges and textures.



Feature Visualization of convolutional network trained on ImageNet from [Zeiler and Fergus 2013]. Source: <https://www.slideshare.net/slideshow/yann-le-cun/27570713>



# Computer Vision: CNN

## Convolutional Layers

- Convolution is a mathematical operation that extract features in an image.
- Averaging + Sliding → Convolution.
- This operation involves “**sliding**” the filter over the image, element-wise **multiplying** the values of the filter, and **summing** the results.
- This process is repeated for every pixel in the image.

1. Multiply the 1st neighbor pixel by the 1st value in the Kernel

|     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|
| 125 | 213 | 98  | 203 | 202 | 170 |
| 104 | 145 | 161 | 204 | 201 | 157 |
| 72  | 8   | 209 | 202 | 194 | 144 |
| 73  | 9   | 202 | 201 | 194 | 156 |
| 81  | 15  | 189 | 185 | 181 | 144 |
| 15  | 189 | 185 | 194 | 227 | 158 |

Original Image

|   |   |   |
|---|---|---|
| 1 | 1 | 1 |
| 1 | 1 | 1 |
| 1 | 1 | 1 |

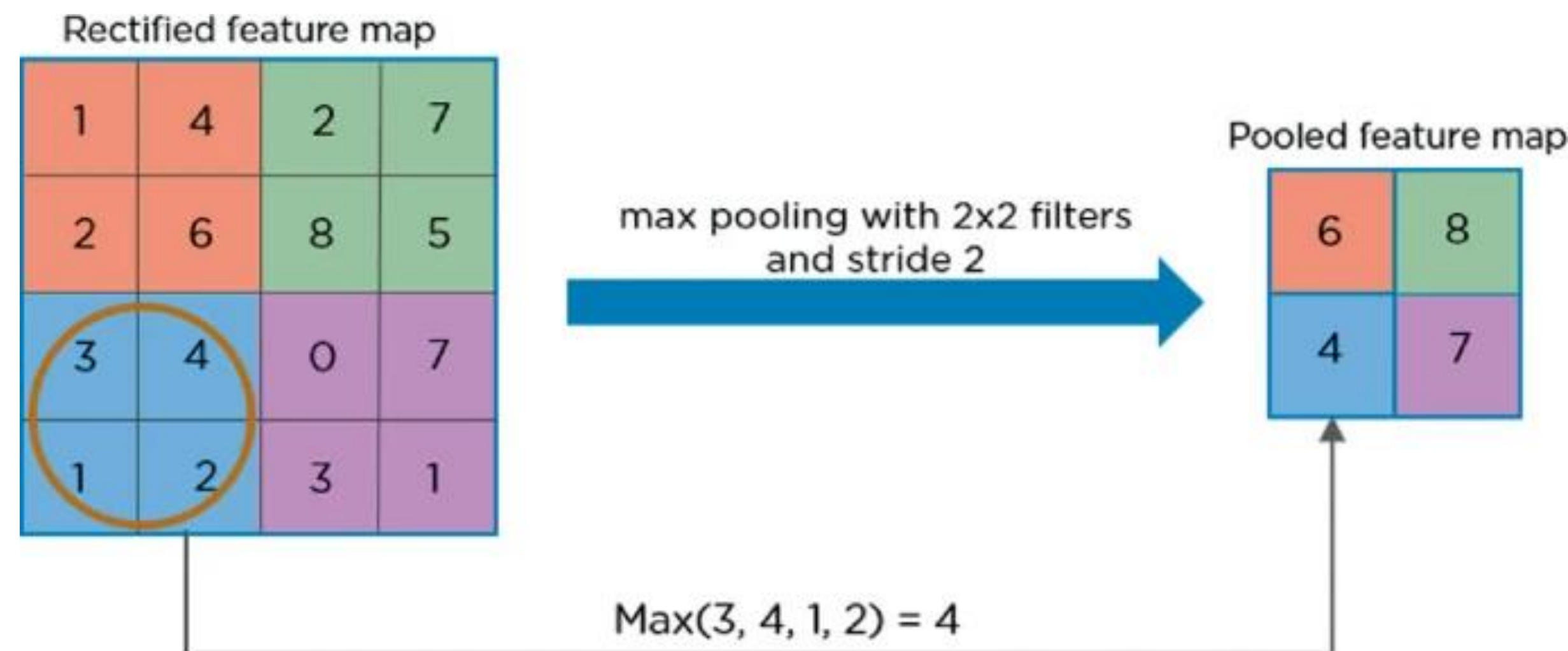
3x3 Box  
Blur Kernel

<https://medium.com/@ChanakaDev/deep-learning-for-computer-vision-8117d7fc4d76>

# Computer Vision: CNN

## Pooling Layers :

- Pooling is a **down-sampling** operation that reduces the dimensionality of the feature map.
- Pooling summarizes local neighborhoods, helping keep the most important activations while reducing computation and sensitivity to small shifts.

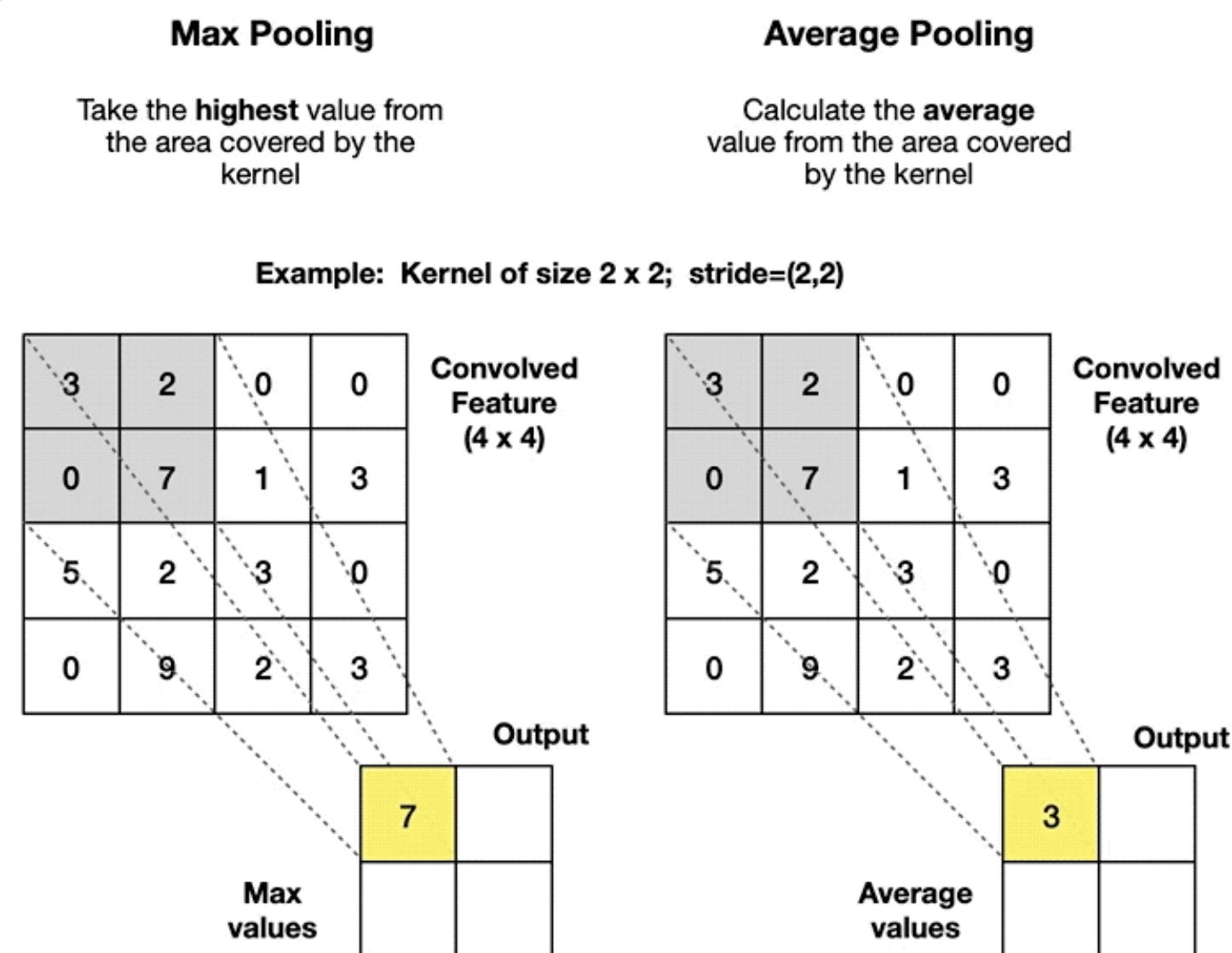


Das, V., Das, D., Hazra, R. (2024). Vernacular Language Handwriting Recognition Using Deep Learning Techniques. In: Gabbouj, M., Pandey, S.S., Garg, H.K., Hazra, R. (eds) Emerging Electronics and Automation. E2A 2022. Lecture Notes in Electrical Engineering, vol 1088. Springer, Singapore. [https://doi.org/10.1007/978-981-99-6855-8\\_46](https://doi.org/10.1007/978-981-99-6855-8_46)

# Computer Vision: CNN

## Pooling Layers :

- Pooling is a **down-sampling** operation that reduces the dimensionality of the feature map.
- Pooling summarizes local neighborhoods, helping keep the most important activations while reducing computation and sensitivity to small shifts.



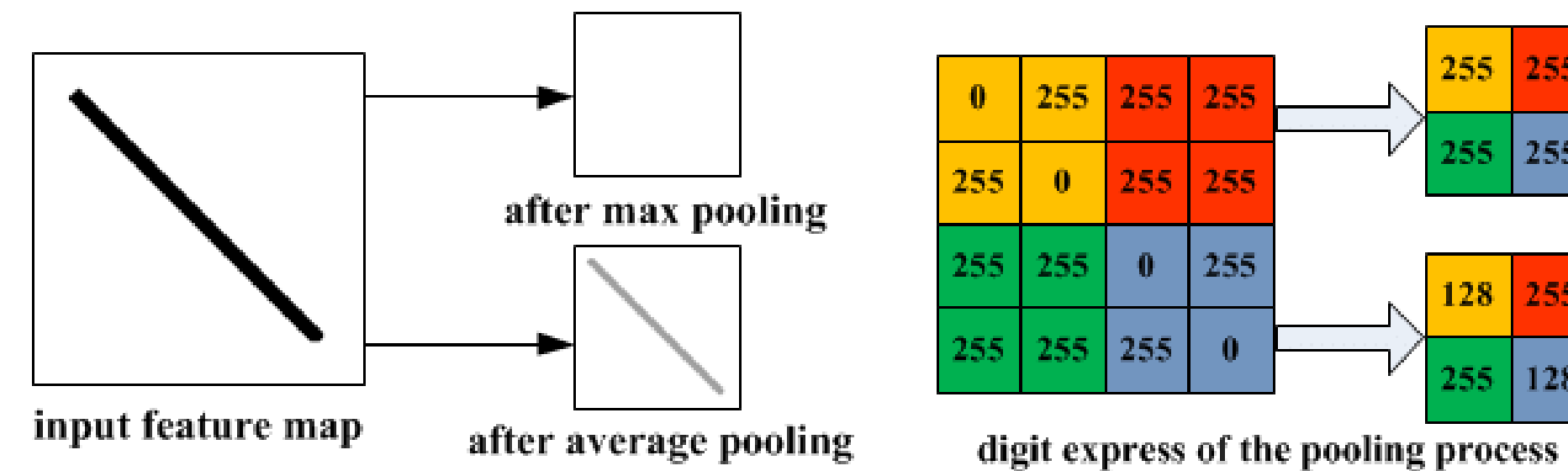
Medium: Intro to Pooling layers in CNN <https://pub.towardsai.net/introduction-to-pooling-layers-in-cnn-dafe61eabe34>



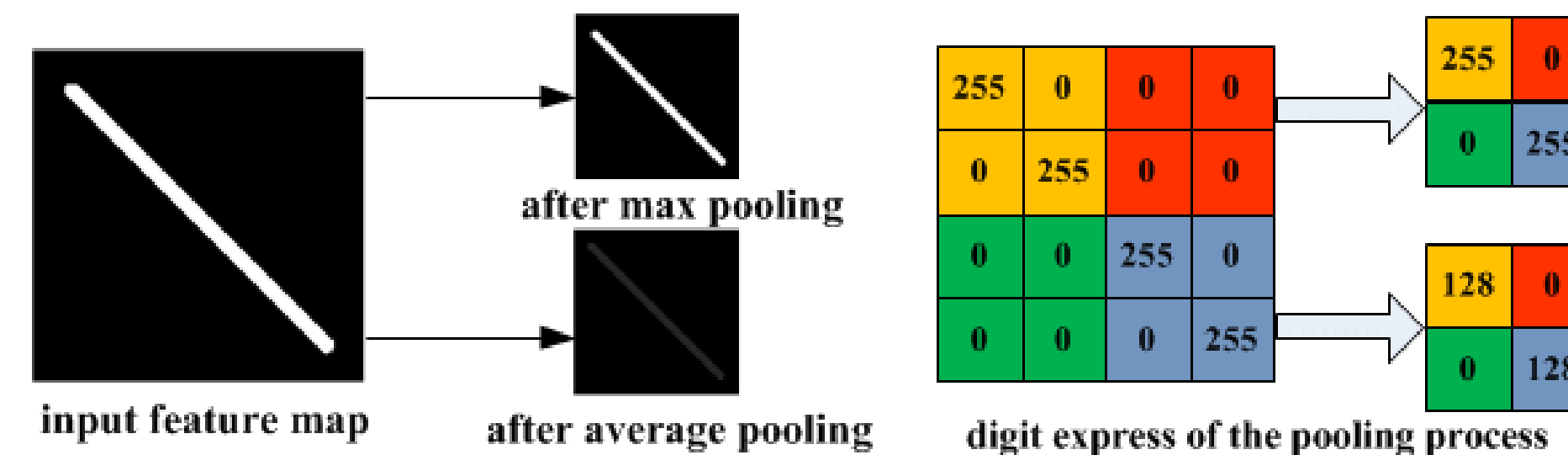
# Computer Vision: CNN

## Pooling Layers :

- Pooling is a **down-sampling** operation that reduces the dimensionality of the feature map.
- Pooling summarizes each local region, for example by taking its maximum or average value.



(a) Illustration of max pooling drawback



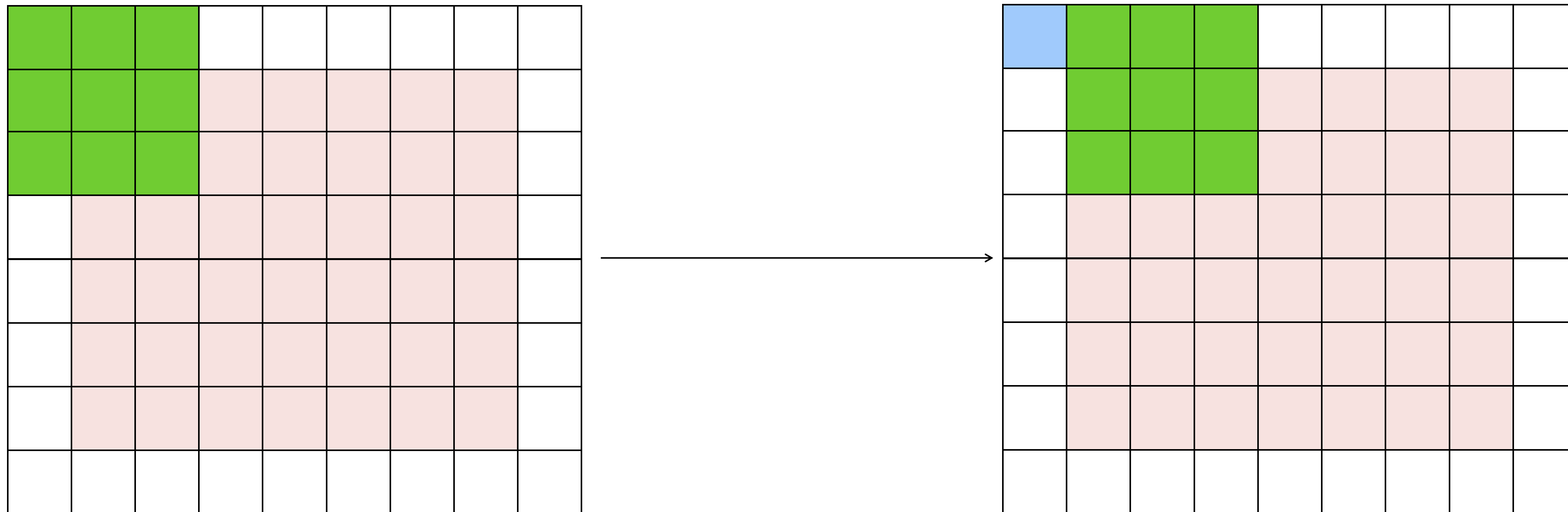
(b) Illustration of average pooling drawback

Yu, Dingjun, Hanli Wang, Peiqiu Chen and Zhihua Wei. "Mixed Pooling for Convolutional Neural Networks." Rough Sets and Knowledge Technology (2014).

# Computer Vision: CNN

## Padding :

Sometimes, we expand the size of the input (or the previous layer) but treat the newly added units as having a fixed value (typically, zero) in order to **preserve information near the image boundaries**.

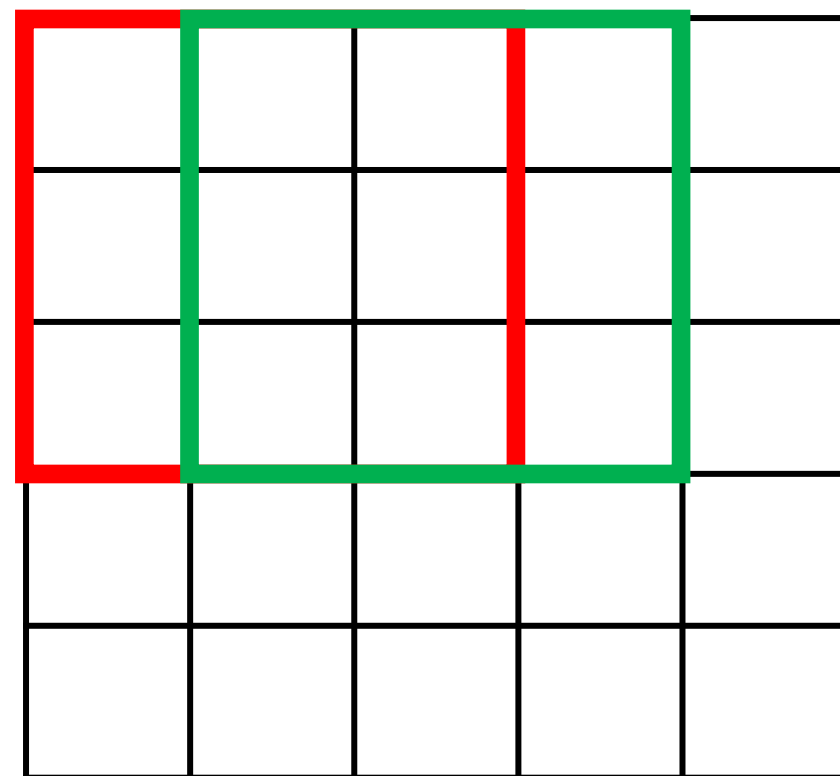


# Computer Vision: CNN

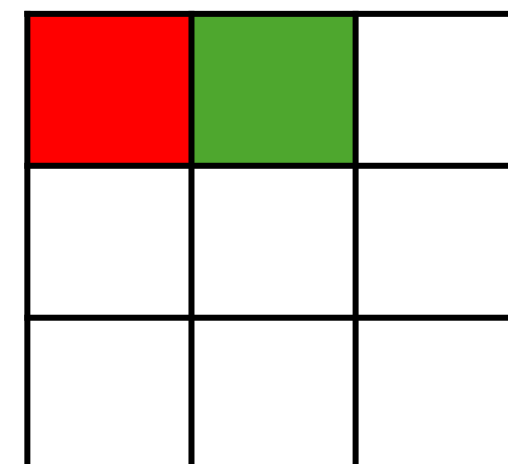
**Stride** refers to the step size with which a convolutional filter moves.

- A **stride of 1** means the filter moves one pixel at a time, leading to a highly detailed feature map with minimal size reduction.
- A **stride of 2** means the filter moves two pixels at a time, reducing the spatial dimensions more aggressively.

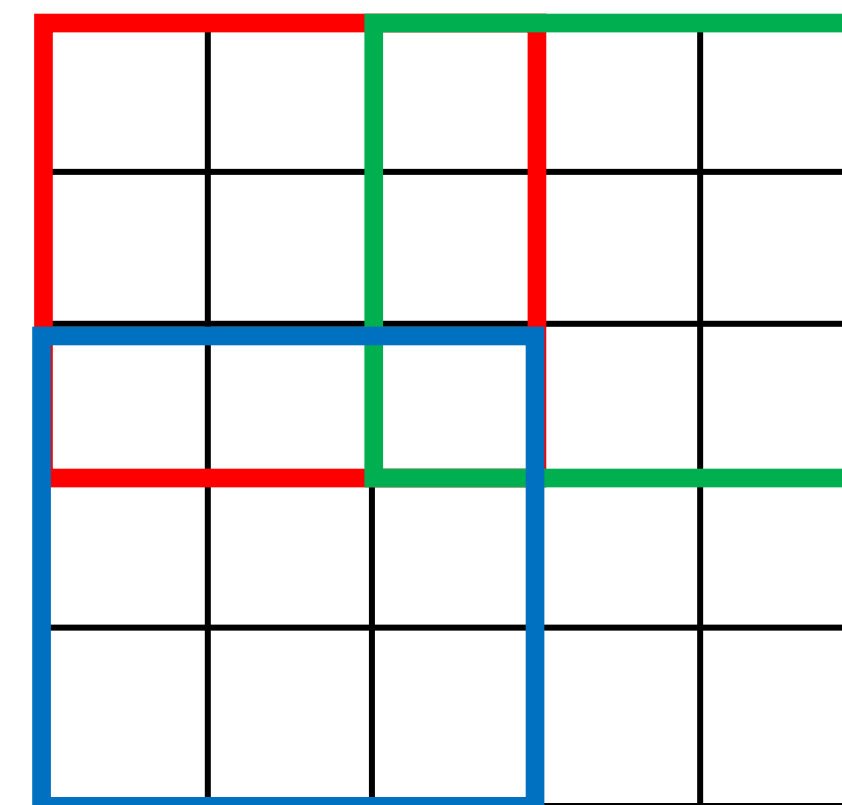
Convolution with stride =1



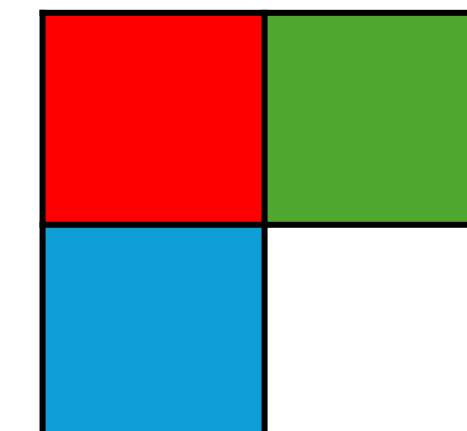
Output



Convolution with stride =2



Output

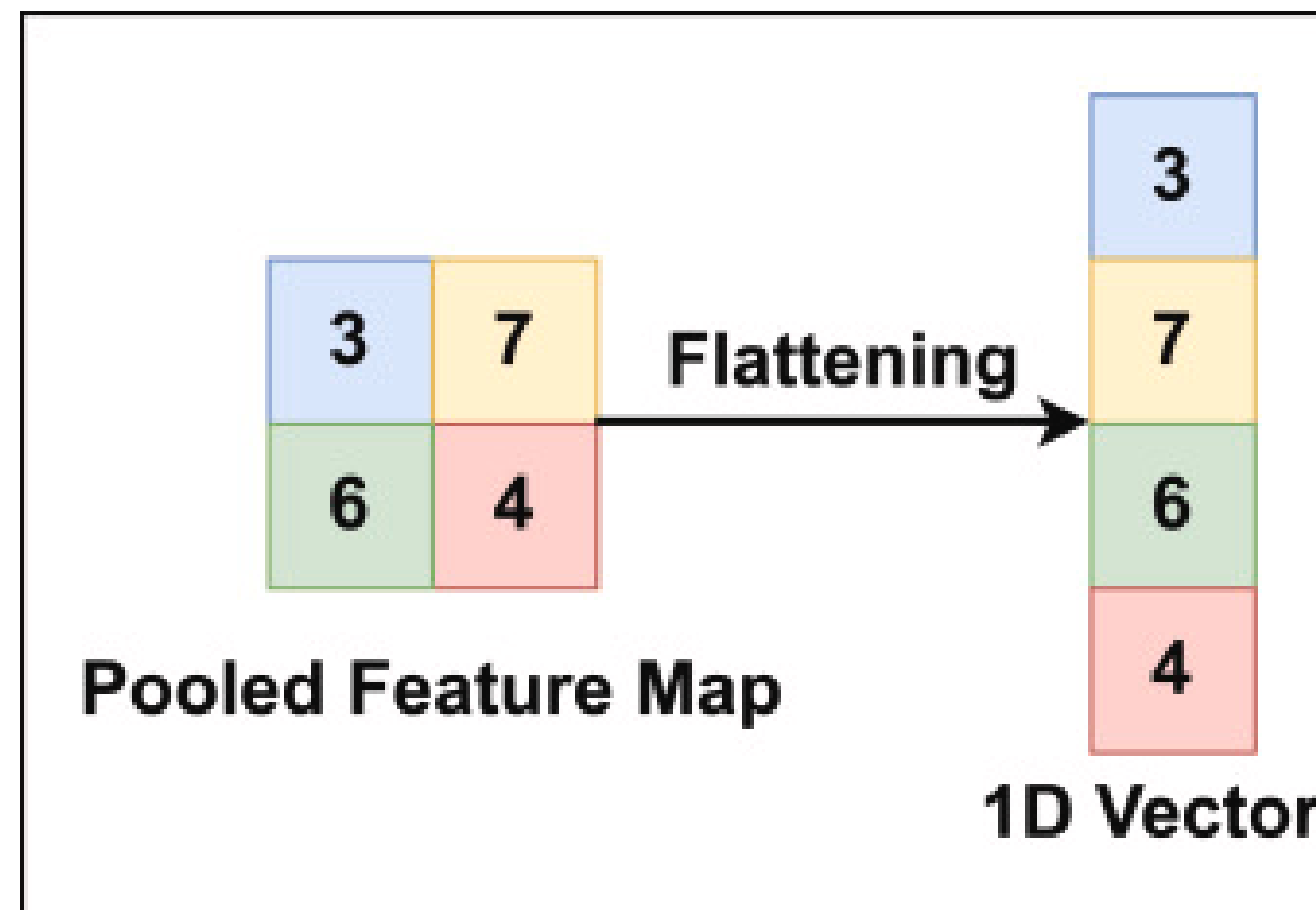




# Computer Vision: CNN

## Flattening Layer :

- After the convolution and pooling operations, the feature maps still exist in a multi-dimensional format.
- Flattening converts these feature maps into a one-dimensional vector so the features can be passed to fully connected layers for classification or regression.



# Computer Vision: CNN

- For an RGB image, there are separate kernels for each colour channel.
- Each filter produces a separate feature map.
- The output normally has multiple channels, and each channel is a feature map produced by a particular kernel.

Grayscale

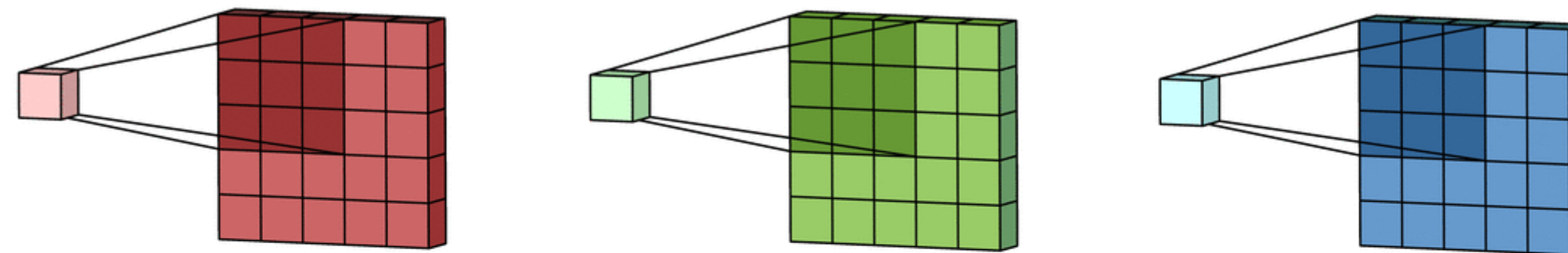
|     |     |     |
|-----|-----|-----|
| 255 | 150 | 0   |
| 39  | 150 | 221 |
| 255 | 150 | 0   |

RGB

|     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|
| 255 | 150 | 0   |     |     |     |     |
| 39  | 150 | 221 | 150 | 0   |     |     |
| 255 | 150 | 0   | 150 | 255 | 70  | 0   |
|     |     | 200 | 150 | 0   | 70  | 255 |
|     |     |     |     | 255 | 180 | 0   |

# Computer Vision: CNN

- For an RGB image, there are separate kernels for each colour channel.
- Each filter produces a separate feature map.
- The output normally has multiple channels, and each channel is a feature map produced by a particular kernel.



<https://blog.devgenius.io/convolutional-neural-networks-98e2067072a9>



# Computer Vision: VGG16

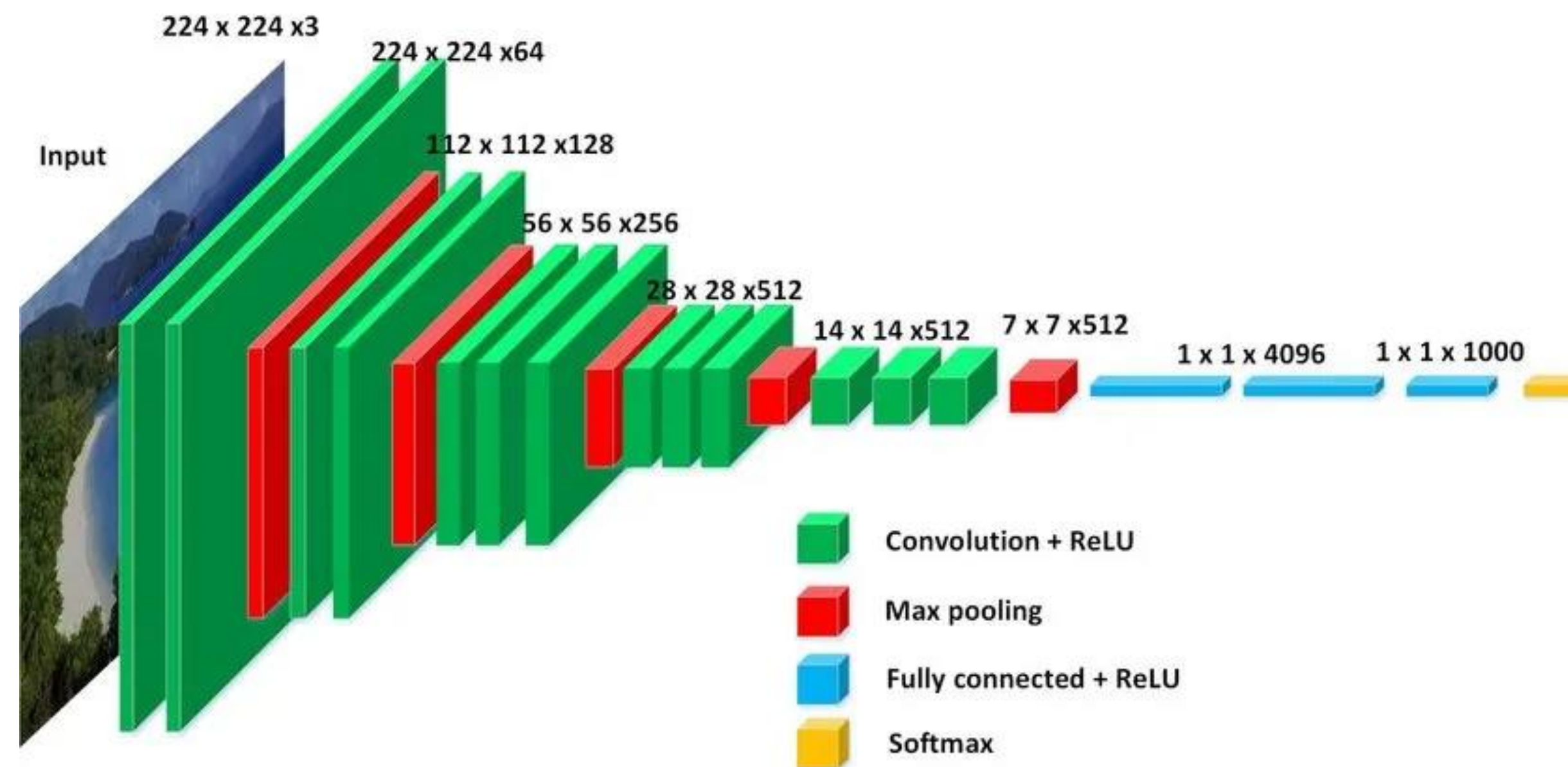
- **VGG16** is a CNN architecture developed by the Visual Geometry Group at the University of Oxford (Simonyan and Zisserman, 2014).
- It has **16 learnable layers: 13 convolutional layers**, 3 fully connected layers, with 5 max-pooling layers in between.
- The architecture is characterized by the use of small 3x3 convolution kernels and ReLU activation functions.



<https://smuhabdullah.medium.com/understanding-vgg16-a-powerful-deep-learning-model-for-image-recognition-d40b074fd01c>

# Computer Vision: VGG16

- It has 16 learnable layers: 13 convolutional layers and 3 fully connected layers, with 5 max-pooling layers in between.
- The architecture is characterized by the use of small 3x3 convolution kernels and ReLU activation functions.

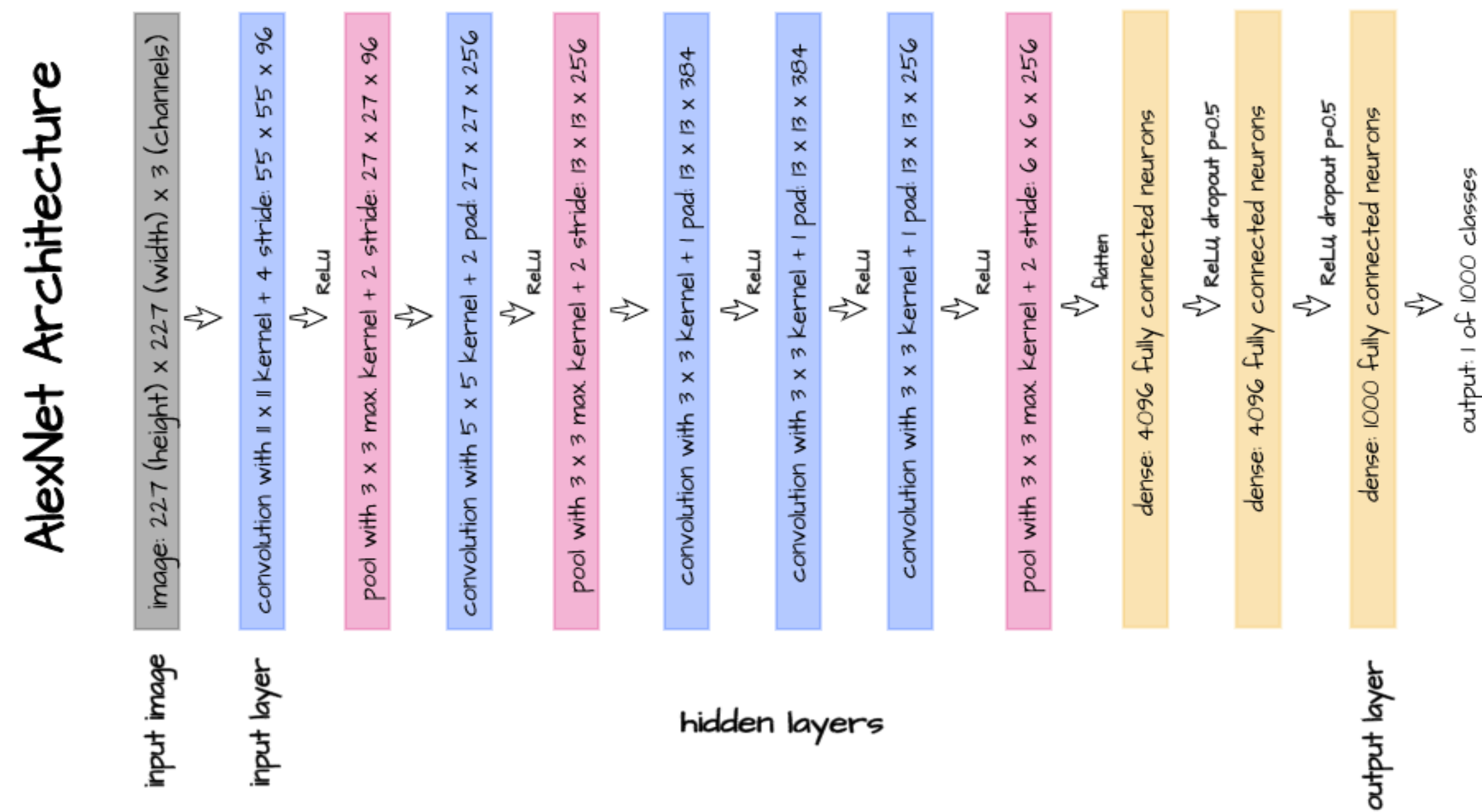


<https://smuhabdullah.medium.com/understanding-vgg16-a-powerful-deep-learning-model-for-image-recognition-d40b074fd01c>



# Computer Vision: AlexNet

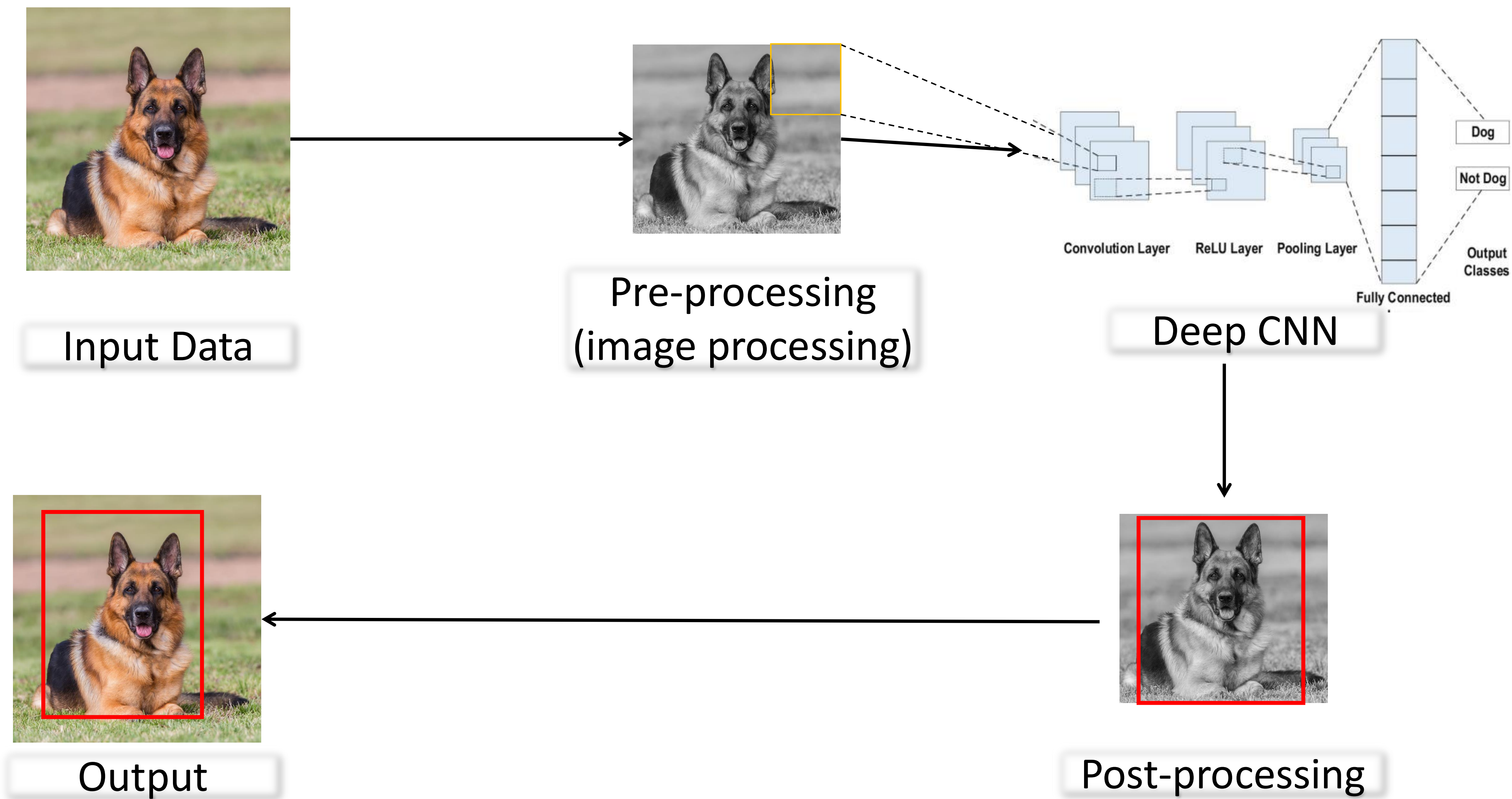
- **AlexNet** is a CNN architecture developed for image classification tasks, notably achieving prominence through its performance in the ImageNet Large Scale Visual Recognition Challenge (ILSVRC).
- It has **8 layers (5 conv + 3 fully connected)** and became practical because it was trained using GPU acceleration.



<https://blog.paperspace.com/understanding-interpreting-convolutional-neural-network-architectures/>



# A Practical Vision Pipeline



# Roadmap: Why Cognitive Vision Matters

- Introduction
- Image Processing
  - Histogram Equalization
  - Noise Removal
  - Edge Detection
- Computer Vision
  - Convolutional Neural Networks
    - ✓ VGG16
    - ✓ AlexNet
- **Cognitive Vision**

# Cognitive Vision: Principles

Is perception only about reconstructing a 3D world from images, or does it also require reasoning, learning, and prediction?

- Traditional computer vision often focuses on building descriptions of the world by identifying objects, actions, shapes, and structure.



# Cognitive Vision: Principles

- In many AI systems, perception is treated as a **black box** that just turns raw images into labels.
- The system often uses static data such as images or single video frames and does not really "**understand**".
- Moving from raw pixel data to **high-level symbolic understanding** (e.g., "*John is picking up the red ball to throw it*") is not easy.
- There's often no causal link — the system **doesn't understand why things happen**, or how the present relates to the past.
- Because of this, such systems struggle to **anticipate the future**.

# Cognitive Vision: Principles

Is perception only an inference process?

- Additional knowledge is needed through inference – as we look at the world, we think about it.

Cognitive vision continuously exchanges information between perception and reasoning.

Actions are guided by perceptual expectations: if Jack through the ball toward me, should I move?

# Cognitive Vision: Vision and Reasoning Interaction

- **Cognitive vision** goes beyond extracting features such as edges or shapes; it aims to **understand and reason** about what's happening in the scene
  - The system **predicts** what it expects to see or sense.
  - Then it **explores** to check if its predictions are correct.
- This **prediction–exploration loop** helps the system go from just labeling things to truly interacting with and learning about the world, like humans do.
- So, **cognitive vision** is about combining **perception with reasoning, learning, and action**, enabling systems to adapt, anticipate the future, and deal with uncertainty.



# Cognitive Vision: Vision and Reasoning Interaction

- Cognitive vision to support human-robot interaction.
- iCub's behavior is driven only by the direction of the subject's gaze, making explicit the intention to reach for the left or right hand.

Mutual gaze with a robot affects human neural activity  
and delays decision-making processes

Marwen Belkaid\*, Kyveli Kompatsiari\*, Davide De Tommaso,  
Ingrid Zablith, and Agnieszka Wykowska

*Science Robotics*, 2021



# Cognitive Vision: Vision and Reasoning Interaction

- Cognitive Vision is not just about identifying "**what**" is in the scene and "**where**" it is.
- Cognitive vision **combines vision and reasoning** to drive **action**. It's a **multi-modal system**—vision works together with other senses and cognitive functions.
  - The questions extend beyond what and where to why, how, and who.
  - It also asks how visual information can be synthesized to anticipate the effects of actions.

# References

- OpenCV: a real-time optimized computer vision library.  
<https://opencv.org/>
- Nilsson, N. J. (1998). *Artificial intelligence: a new synthesis*. Morgan Kaufmann. Chapter 6.
- Aloimonos, Y., & Sandini, G. Principles of Cognitive Vision. In Cangelosi, A., & Asada, M. (Eds.). (2022). *Cognitive robotics*. MIT Press. Chapter 14.
- Davies, E. R. (2018). *Computer vision: principles, algorithms, applications, learning*. Academic Press.
- Semantic Segmentation of Vegetation in Remote Sensing Imagery Using Deep Learning. September 2022. DOI: 10.48550/arXiv.2209.14364. Alexandru Munteanu, Marian Neagul.

